Supplementary Information

Few biological parameters suffice to reproduce measured dynamics in systems biology models: local sensitivity-based reduction with formally verified mathematical guarantees

Mónica Miranda R., Carlos Salazar, César Martín, Adriana Aguirre

Contents

Supplementary Table S1. Selected biological parameters **2**

Supplementary Table S2. Notation **3**

Supplementary Table S3. Formal proofs **4**

Supplementary Note 1. Verification of the simulator **6**

Supplementary Note 2. Logical sequence of the formal proofs **6**

Setting and goal . . . 6
Part I. Finite-dimensional core (steps 1–13) . . . 9
Part II. Ordinary differential equations (steps 14–18) . . . 10
Part III. Certificates for ODE models (steps 19–22) . . . 10
Part IV. Hypotheses of the benchmark models (steps 23–31) . . . 11
Connection with the numerical results and what the proofs do not cover . . . 12

Supplementary Note 3. Statements of the formal theorems **12**

Part I. Finite-dimensional core of the method (fixed sensitivity matrix) . . . 14
Part II. Ordinary differential equations: existence and sensitivities . . . 21
Part III. Certificates for ODE models . . . 24
Part IV. Hypotheses of the benchmark models (kinetic models and reaction networks) . . . 26

Supplementary Note 4. The verified benchmark models **33**

Agreement of the translation with the SBML models . . . 34
A complete example: Boehm_JProteomeRes2014 . . . 35
Benchmark problems without a verified model . . . 36
Data of each model . . . 36

Supplementary Note 5. Independent reproduction of the verification **49**

What each check establishes . . . 50
Versions . . . 50
Steps . . . 50

Concluding remarks **50**

Supplementary Table S1. Selected biological parameters

Final subset S for each problem (after backward elimination) and the parameters removed by backward elimination. Calibration parameters, which are always re-estimated, are not listed. Crauste and Beer have no admissible subset; the subset at which the method stopped is shown. For Crauste it is not meaningful: about half of the perturbed simulations have no solution on the measured horizon (finite-time growth without bound), so the sensitivity matrix is incomplete.

System	Selected biological parameters S	Removed by backward elimination
Alkan	kt, k_rep_nhej	–
Armistead	k_d, alpha_cer	k00, k2
Bachmann	CISRNADelay, JAK2ActEpo	–
Beer	kdegi_typeIDT1, kdegi_typeIDT3, ksyn_typeIDT3, kdim_typeIDT1, beta_typeIDT3_ExpID2, beta_typeIDT3_ExpID3, beta_typeIDT3_ExpID5, beta_typeIDT3_ExpID1, beta_typeIDT3_ExpID4, init_Bac, beta_typeIDT1_ExpID6, beta_typeIDT1_ExpID5, beta_typeIDT1_ExpID1, beta_typeIDT1_ExpID4, beta_typeIDT1_ExpID3, kdim_typeIDwt, ksyn_typeIDT5, kdim_typeIDT5	–
Bertozzi	R0_NY, I0_NY	–
Blasi	a_basal, a_k16_k12k16	a_k12k16_k8k12k16, a_k5k12_k5k8k12, a_k8k12k16_4ac
Boehm	k_imp_hetero, k_phos	–
Borghans	Kz, K_par	–
Brannmark	k1f, km2	–
Bruno	kb1, init_bcar2, init_zea_1, init_bcar1, init_bcry_1	szea, init_b10_1
Chen	k106, k44	–
Crauste	rho_E	–
Elowitz	n_Hill, tau_mRNA	tau_prot, tau_prot_GFP, eff
Fiedler	tau2, k6	–
Fujita	init_AKT, reaction_6_k1	–
Giordano	alpha_4, alpha_28	–
Isensee	kf_RIIp_C_2__RII_C_2, xi_kf_RII_C_2__RII_2	ks_AC_cAMP, KD_cAMP, xi_AC_cAMP_Fsk, kf_RIIp_cAMP_C_2__RIIp_2, kf_RII_2__RII_C_2
Lang	kDeE2f1, kDeCe	–
Laske	k_imp, k_syn_R_M	k_bind_M1
Liu	kappa, phi	–
Lucarelli	init_Rec, S2tot, S3tot, S_dephosphos, S_phos, geneF_turn, k_234, geneK_turn, S4tot, geneH_turn	–
Okuonghae	transmission_rate_effective, gamma_i	–
Oliveira	beta_0, h_hosp_rate	–
Perelson	c, delta	–
Rahman	infected_moderate_transmission_rate, infected_normal_worsen_rate	–
Raia	SOCS3mRNA_production, init_Rec_i	–

System	Selected biological parameters S	Removed by backward elimination
Raimundez	kdeg_membran__MKN1, d_kimp_pEGFR_EGF_2__MKN1_2_HS746T	ka_MAPK__MKN1
SalazarCavazos	ratio_kpkd_Y1068__FREE, ratio_kpkd_YN__FREE	–
Schwen	ini_R1, ini_R2fold	–
Smith	k8, kminus7b	–
Sneyd	l4, l2	–
Weber	s12, a21	–
Zhao	R_Stage_I_China, R_Stage_II_Wuhan, gamma_1_Stage_I_Hubei	–
Zheng	k01_02, k13_12	–

Supplementary Table S2. Notation

Symbol	Meaning	Value used
$x(t;\theta) \in \mathbb{R}^n$	state trajectory of the model $\dot{x} = f(x, \theta)$	–
$\theta \in \mathbb{R}^p$, θ_0	parameter vector; nominal (published) value	PEtab nominal values
Θ_B, Θ_C	biological parameters (in the SBML model) and calibration parameters (observation scalings and offsets)	from the PEtab tables
$y_i(\theta) = h(g_i(x(t_i), \theta))/\sigma_i$	i-th scaled output: observable g_i, transformation h (lin, log, $\log_{10}$), noise s.d. σ_i, at the condition and time of measurement i	PEtab measurement table
$J = \partial y / \partial \log \theta \in \mathbb{R}^{m \times p}$	relative sensitivity matrix (central differences, step δ)	$\delta = 0.01$
$E_j = \|J_{:,j}\|^2$	sensitivity energy of parameter j	–
$R_{\text{var}}(S)$	fraction of the energy of biological parameters captured by S	$\tau_R = 0.89$
s_j	finite-perturbation effect of parameter j (stage-2 ordering)	$\eta = 0.1$
$\kappa(S)$, VIF(S)	condition number and largest variance inflation factor of the normalised columns of J_S	$\tau_\kappa = \tau_{\text{VIF}} = 10$
Δy_{full}, Δy_S	output change when all parameters, or only those in $S \cup \Theta_C$, take their scenario values	$N = 15$, $\varepsilon = 0.05$
$\cos \Delta$	cosine similarity between Δy_{full} and Δy_S	–
e_{rel}	relative error without re-estimation	–
e_{fit}	relative error after re-estimating θ_S and θ_C	–
$\tau_e = \sqrt{1 - c^2}$	admissibility threshold on the median e_{fit}	$c = 0.9$, $\tau_e = 0.436$
$k_{\min}$	minimum subset size	2
χ^2/n	mean squared noise-scaled residual on test data	–
e_{pred}	relative prediction error with respect to a known truth (synthetic experiment)	–

Supplementary Table S3. Formal proofs

All formal proofs are collected in the folder `lean_proofs/` of the repository, a self-contained Lean 4 project (Lean 4.23.0, Mathlib v4.23.0) that compiles with a single command (`./verify.sh`). It contains 31 theory modules with 289 theorems and lemmas, and 28 benchmark models translated from SBML. No file contains `sorry` or `axiom`, and `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound` for every theorem. Modules are listed in the logical order of the proofs (Supplementary Note 2): # is the step number, and every module only uses modules with a smaller number (column 'Uses'). n is the number of theorems and lemmas in the module (its size, not an index). Supplementary Note 3 gives the statement and meaning of every theorem and lemma; the file `THEOREMS.md` in the folder lists them as well.

#	Module	n	Uses	Content	Main theorems
Part I. Finite-dimensional core of the method (fixed sensitivity matrix)					
1	`FirstOrderError Bound`	12	–	Energy $E_j = (J^\top J)_{jj}$, R_{var} and first-order output error when parameters are fixed	`total_eq_kept_add_ removed, first_order_ error_from_Rvar`
2	`TrajectoryError Bound`	9	1	From the Jacobian to the trajectory: error bound when discarded parameters are fixed	`sqrt_removedParamSq_le`
3	`ScaledError`	5	–	Theorem 1: the minimum relative error after rescaling equals $\sqrt{1-\cos^2\Delta}$	`min_rel_error_eq`
4	`CosineCertificate`	5	2	Non-asymptotic bounds for finite perturbations; per-scenario certificate of $\cos\Delta$	`finite_trajectory_error, finite_cos_certificate`
5	`RefitMonotone`	5	–	Re-estimating more parameters never increases the fitting error; $e_{\mathrm{fit}} \le e_{\mathrm{rel}}$	`fitErr_anti, fitErr_le_ erel, admissible_mono`
6	`Poda`	2	–	Theorem 2: backward elimination returns an inclusion-minimal admissible subset	`prune_spec, exists_ minimal_admissible`
7	`Identifiability Conditioning`	10	–	κ and VIF bounds imply unique and stable least-squares estimates	`ls_unique, ls_param_ kappa_stability, ls_vif_ inverse_stability`
8	`Spectral Conditioning`	22	7	κ and VIF exactly as computed numerically (spectral theorem, matrix inverse)	–
9	`GreedySelection`	7	7	Greedy selection returns a subset with $\kappa \le \tau_\kappa$ and VIF $\le \tau_{\mathrm{VIF}}$; permanent discard is correct	`greedy_kappa_vif_ guarantee`
10	`PatternMechanisms`	4	2, 9	Opposing effects imply rejection by κ and non-identifiability; zero discarded energy implies exact first-order reduction	`not_okKappa_of_kernel, exact_reduction_of_zero_ energy`
11	`Numerical Robustness`	6	4, 9	Finite-difference error; R_{var}, κ and VIF certified from approximate values with margins	`fd_column_error, rvar_ certified_from_approx`
12	`LinearizedFit`	3	3	The first-order e_{fit} differs from the exact one by a second-order term	`lin_le_scaled, fit_ lower_of_lin`

#	Module	n	Uses	Content	Main theorems
13	`OutputComposition`	4	–	Error bounds transfer from states to measured outputs, including log and $\log_{10}$	`output_uniform_bound`, `log10_lipschitz`
Part II. Ordinary differential equations: existence and sensitivities					
14	`GlobalLipschitzODE`	3	–	Global existence on $[a, b]$ for globally Lipschitz vector fields	–
15	`ParamDiffODE`	8	14	Solutions depend differentiably on parameters and initial values; the derivative solves the variational equation	`hasFDerivAt_solution_param_init`
16	`LocalExistence`	9	15	Solutions exist for parameters near θ_0 when the nominal solution exists	–
17	`Sensitivity Lipschitz`	8	–	Explicit Lipschitz constants of trajectories and sensitivities in θ (Grönwall)	`sens_lipschitz, traj_lipschitz`
18	`EventSystems`	1	16	Systems with events at fixed times: existence and differentiability across segments	–
Part III. Certificates for ODE models					
19	`CertifiedODE Reduction`	1	2, 15	Local certificate: $R_{\mathrm{var}}(S) \geq r$ bounds the trajectory error of the reduced model	`certified_parameter_reduction`
20	`CertifiedFiniteODE`	5	4, 17, 19	Certificate of $\cos \Delta$ for the ODE with an explicitly bounded Lipschitz constant	`certified_cos_ode`
21	`ExactLinear Certificate`	13	20	Certificates with the exact linear error; local version without Lipschitz constants	`finite_cos_certificate_exact, certified_cos_ode_exact`
22	`Nominal Certificate`	1	16, 21	Local $\cos \Delta$ certificate from the nominal trajectory only	–
Part IV. Hypotheses of the benchmark models (kinetic models and reaction networks)					
23	`KineticRegularity`	11	22	Rate laws (mass action, Michaelis–Menten, Hill, ...) define smooth vector fields on their domain	–
24	`Positivity Invariance`	15	23	Quasi-positive models keep concentrations non-negative	–
25	`GlobalExistence`	11	14, 24	Global existence under linear growth on the non-negative orthant	–
26	`StrictExistence`	12	25	Global existence with strictly positive concentrations	`exists_global_solution_strict`
27	`KineticCheck`	19	18, 24	Verified automatic checkers for translated rate laws	–
28	`KineticNetwork`	18	18, 25, 27	Reaction networks: complete check including global existence	`network_final`
29	`RiccatiNetwork`	9	28	Existence on a finite horizon under quadratic growth (Riccati bound; used for Crauste)	`riccati_final`
30	`StrictNetwork`	46	26, 28	Networks with strictly positive domains (e.g. $\beta SI/N$, Hill with real exponent)	`strict_final_init`

#	Module	n	Uses	Content	Main theorems
31	`IntervalInit`	5	30	Parameter-dependent initial values certified with rational interval arithmetic	`strict_final_initI`
	Total	289			

Verified benchmark models (28). Armistead, Bachmann, Bertozzi, Blasi, Boehm, Borghans, Brannmark, Bruno, Chen (501 states, in four parts), Crauste (horizon $T \leq 1$), Elowitz, Fiedler, Froehlich (1,228 states, in twelve parts), Fujita, Giordano, Isensee, Lang, Laske, Lucarelli, Okuonghae, Rahman, Raia, Raimundez, SalazarCavazos, Sneyd, Weber, Zhao and Zheng (folder `lean_proofs/Models/`). For each model, the final theorem establishes that the vector field is smooth on its domain, that the solution exists on the simulated horizon and that concentrations remain non-negative. The data of each model and the checks executed are given in Supplementary Note 4 and Table S6, which also explain why the remaining seven benchmark problems were not verified.

Supplementary Note 1. Verification of the simulator

Our simulation of the measured outputs was compared, at the nominal parameters, with the reference simulations distributed with the benchmark, matching each measurement by observable, condition and time. For the problems with comparable reference files the median relative difference was below 10^{-5} in all but five cases: Chen (1.1×10^{-4}), Zheng (whose reference file contains the measured data rather than a simulation), Crauste (0.8%, very stiff model), Lang (2.8%, an oscillating cell-cycle model simulated to $t = 136{,}472$, where peaks differ by an accumulated phase shift) and Perelson (17%; the initial state of the model is stationary and our simulation respects it, which suggests that the reference was generated with different values). Lang and Perelson are therefore not close agreements with their references; for Lang the difference is an accumulated phase shift of an oscillator over a long horizon, and for Perelson our simulation is consistent with the equations and the stationary initial state of the SBML model, so the discrepancy most likely lies in the reference. Both models were kept in the analysis, and their results should be read with this caveat. Several problems have no comparable reference (no file, or different condition identifiers). The full comparison is given in the repository (`certificados/resultados/salidas/VERIFICACION_SIMULADOR.md`).

Supplementary Note 2. Logical sequence of the formal proofs

This note explains, in mathematical terms, the chain of results that the Lean code proves, in the order in which they build on each other. The order is that of the import graph of `lean_proofs/`: step k only uses steps with a smaller number (Supplementary Table S3, column 'Uses'). Lean names of the main results are given in `typewriter` font; the statement and meaning of every theorem are in Supplementary Note 3, and the notation in Supplementary Table S5. Supplementary Figure S1 shows how the formal and the numerical parts of the work fit together, and Supplementary Table S4 states the scope of each formal guarantee.

Setting and goal

A model is an ODE $\dot{x} = f(x, \theta)$ on $[0, T]$ with $x(0) = x^0(\theta)$, $\theta \in \mathbb{R}^p$, sampled at times $t_1, \ldots, t_N$; the sampled (noise-scaled) outputs form a vector $F(\theta) \in \mathbb{R}^m$. Fixing the parameters outside a subset S

at their nominal values means replacing a perturbation $\Delta\theta$ by $\Delta\theta_S$, whose components outside S are zero. The proofs establish five guarantees:

(G1) for small perturbations, the output error of the reduced model, $\|F(\theta_0 + \Delta\theta) - F(\theta_0 + \Delta\theta_S)\|$, is bounded by the sensitivity energy discarded, i.e. by $R_{\mathrm{var}}(S)$;

(G2) the cosine $\cos\Delta$ between full and reduced output changes determines the best relative error obtainable by rescaling the reduced response, $\sqrt{1-\cos^2\Delta}$, and the linearised re-estimation of several parameters is at least as good;

(G3) the conditioning tests (κ, VIF) imply that the *linearised* least-squares problem for θ_S has a unique solution that is stable with respect to the data;

(G4) the fitting error is monotone in the re-estimated set, and backward elimination returns a subset from which no single parameter can be removed;

(G5) the hypotheses needed for (G1)–(G3) (smoothness, existence of solutions, positivity, differentiability) hold for the translated benchmark models.

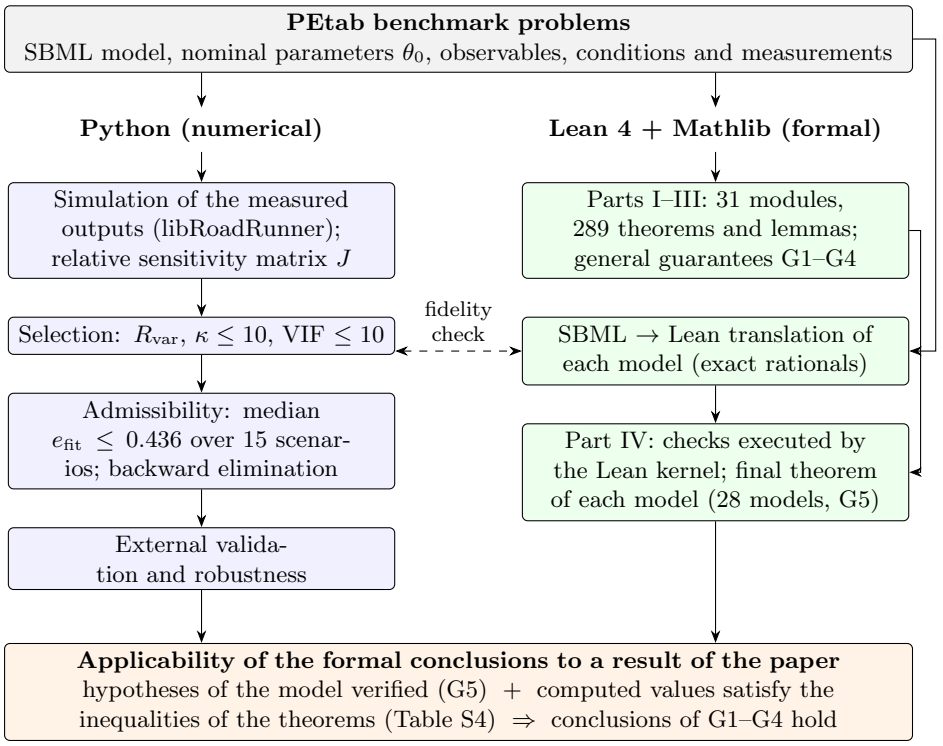
Parts I–III prove (G1)–(G4) for any model satisfying explicit hypotheses; Part IV proves that the 28 verified benchmark models satisfy them. Supplementary Table S4 states, for each guarantee, what is proved, what is not, and which numerical quantity connects it with the results of the paper.

Supplementary Table S4. Scope of the formal guarantees. Numerical values refer to the final subsets of the 32 admissible benchmark problems and were computed in Python in double precision.

Guarantee	What Lean proves	What it does not prove	Numerical connection with the paper
G1. Error of the reduced model	For small $\Delta\theta$: $\|F(\theta_0+\Delta\theta) - F(\theta_0+\Delta\theta_S)\| \le (\sqrt{(1-R_{\mathrm{var}})E}+\eta)\|\Delta\theta\|$; an explicit version for finite $\|h\| \le \rho_0$ with a Lipschitz constant L_c (steps 1–4, 19–20)	A small error for arbitrarily large perturbations; that the selected subsets have R_{var} close to 1	$R_{\mathrm{var}}(S)$ of the final subsets has median 0.56 and is ≥ 0.89 in 2 of 32; the bound is therefore informative only for some subsets, and admissibility is decided by e_{fit} (G2, G4); the explicit finite version could not certify any benchmark scenario because L_c is very large (see below)
G2. Cosine and fitting error	$\min_\alpha \|a-\alpha b\|/\|a\| = \sqrt{1-\cos^2(a,b)}$; the linearised fit of several parameters is no worse; certificates of $\cos\Delta$ for finite and small perturbations (steps 3–4, 12, 20–22)	That the nonlinear multi-parameter optimisation performed in Python reaches its optimum; the value of e_{fit} itself	Median e_{fit} over 15 scenarios at $\pm5\%$ is $\le \tau_e = 0.436$ in all 32 admissible problems (largest value 0.4355); the median $\cos\Delta$ is ≥ 0.9 in 18 of them, so a subset can be admissible because re-estimating several parameters fits better than rescaling
G3. Conditioning	Uniqueness and stability of the linear least-squares estimate with normalised sensitivity columns, with $\kappa = \sqrt{\lambda_{\max}/\lambda_{\min}}$ and $\mathrm{VIF}_j = [(Z^\top Z)^{-1}]_{jj}$ as computed (steps 7–9)	Global or structural identifiability of the nonlinear model; uniqueness of the nonlinear calibration	$\kappa \le 6.6$ and $\mathrm{VIF} \le 9.85$ for all 32 final subsets (thresholds $\tau_\kappa = \tau_{\mathrm{VIF}} = 10$)

Supplementary Table S4. Scope of the formal guarantees. Numerical values refer to the final subsets of the 32 admissible benchmark problems and were computed in Python in double precision.

Guarantee	What Lean proves	What it does not prove	Numerical connection with the paper
G4. Re-estimation and pruning	e_{fit} never increases when parameters are added; $e_{\text{fit}} \leq e_{\text{rel}}$; pruning returns an admissible subset that is minimal with respect to inclusion (steps 5–6)	That the result is the admissible subset of smallest size; that the numerical evaluation of admissibility is exact	Admissibility of each candidate subset is evaluated numerically (15 scenarios, median $e_{\text{fit}} \leq 0.436$)
G5. Benchmark models	For 28 translated models: smooth vector field, existence of the solution on the horizon (every T, or $T \leq 1$ for Crauste), non-negativity, solutions for θ near θ_0 and their differentiability (steps 23–31)	That the SBML and PEtab files are correct; the accuracy of the numerical simulations; experimental conditions that override parameters or initial values (the translation uses the nominal values)	The translated vector fields, parameter values and initial values agree with the SBML models evaluated by libRoadRunner (Supplementary Note 4)



Supplementary Figure S1. Architecture of the formal and numerical verification. Python (left) performs all computations on the PEtab problems; Lean 4 (right) proves the general theorems (Parts I–III, guarantees G1–G4) and checks the hypotheses of each translated model (Part IV, G5). The translation of each model into Lean is compared independently with the SBML model (dashed arrow; Supplementary Note 4). The formal conclusions apply to a result of the paper when the hypotheses of the model are verified and the numerical quantities computed in Python satisfy the inequalities that the theorems require (bottom).

Part I. Finite-dimensional core (steps 1–13)

Here the sensitivity matrix $J = DF(\theta_0)$ is a fixed $m \times p$ matrix with column energies $E_j = \sum_i J_{ij}^2$, $E = \sum_j E_j$, $E_S = \sum_{j \in S} E_j$.

Steps 1–2 (error of the reduced model, G1). The total energy splits as $E = E_S + E_{S^c}$, and by Cauchy–Schwarz

$$\|J\Delta\theta - J\Delta\theta_S\| = \Big\|\sum_{j \notin S} J_{\cdot j}\Delta\theta_j\Big\| \le \sqrt{E_{S^c}}\,\|\Delta\theta_{S^c}\| \le \sqrt{(1 - R_{\mathrm{var}}(S))\,E}\,\|\Delta\theta\|$$

(`first_order_error_from_Rvar`). If F is differentiable at θ_0, the nonlinear error differs from the linear one by $o(\|\Delta\theta\|)$, so for every $\eta > 0$ and small $\Delta\theta$, $\|F(\theta_0+\Delta\theta) - F(\theta_0+\Delta\theta_S)\| \le (\sqrt{(1-r)E}+\eta)\|\Delta\theta\|$ whenever $R_{\mathrm{var}}(S) \ge r$ (`trajectory_error_from_Rvar`). This is the guarantee behind stage 1 of the selection. It is local, and it does not by itself make a subset admissible: admissibility is a separate, numerical test on finite perturbations (steps 5–6).

Step 3 (cosine and fitting error, G2). For vectors a (full response) and b (reduced response),

$$\min_{\alpha \in \mathbb{R}} \frac{\|a - \alpha b\|}{\|a\|} = \sqrt{1 - \cos^2(a, b)},$$

attained at $\alpha^* = \langle a, b\rangle / \|b\|^2$ (`min_rel_error_eq`, Theorem 1 of the paper). Hence, for $c > 0$, a relative error of at most $\tau_e = \sqrt{1 - c^2}$ after rescaling is equivalent to $|\cos\Delta| \ge c$; in particular $\cos\Delta \ge c$ implies it (the sign matters only because α may be negative). The identity concerns a single scale factor. The method re-estimates several parameters, and step 12 shows that the linearised fit over the span of their sensitivity columns is at least as good as rescaling (`lin_le_scaled`); the exact nonlinear fit differs from the linearised one by a second-order term. Thus $\cos\Delta \ge c$ is a sufficient condition for admissibility to first order, not a necessary one, and the actual e_{fit} is computed numerically.

Step 4 (finite perturbations). If DF is L_c-Lipschitz on the ball $\bar{B}(\theta_0, \rho_0)$, Taylor's theorem gives $\|F(\theta_0 + h) - F(\theta_0) - Jh\| \le L_c\|h\|^2$, hence

$$\|F(\theta_0 + h) - F(\theta_0 + h_S)\| \le \sqrt{E_{S^c}}\,\|h_{S^c}\| + 2L_c\|h\|^2,$$

and, writing $A = \|Jh\| - L_c\|h\|^2 > 0$, the inequality $(\sqrt{E_{S^c}}\|h_{S^c}\| + 2L_c\|h\|^2)^2 \le (1 - c^2)A^2$ certifies $\cos\Delta(h) \ge c$ for that perturbation (`finite_cos_certificate`).

Steps 5–6 (re-estimation and pruning, G4). With $e_{\mathrm{fit}}(S) = \inf\{r(\theta) : \theta_j = \theta_{0,j}\ \forall j \notin S\}$, enlarging S (or adding calibration parameters) never increases e_{fit}, and $e_{\mathrm{fit}} \le e_{\mathrm{rel}}$ (`fitErr_anti`, `fitErr_le_erel`); therefore admissibility is preserved under enlargement. Backward elimination stops at an admissible $R \subseteq S$ such that $R \setminus \{x\}$ is not admissible for any $x \in R$ (`prune_spec`, Theorem 2 of the paper).

Steps 7–9 (conditioning and selection, G3). Let Z have the normalised sensitivity columns of S. If $\sigma_{\min}\|u\| \le \|Zu\| \le \sigma_{\max}\|u\|$ with $\sigma_{\min} > 0$, the linear least-squares estimate (the first-order approximation of the re-estimation of θ_S) is unique and, for two data vectors y, y',

$$\|v - v'\| \le \kappa\,\|y - y'\|, \qquad |v_j - v'_j| \le \sqrt{\mathrm{VIF}_j}\,\|y - y'\|$$

(`ls_unique`, `ls_kappa_stability`, `ls_vif_inverse_stability`). Step 8 identifies these quantities with what is computed numerically: $\kappa = \sqrt{\lambda_{\max}/\lambda_{\min}}$ from the eigenvalues of $Z^\top Z$ and $\mathrm{VIF}_j = [(Z^\top Z)^{-1}]_{jj}$ (`ls_kappa_stability_spectral`, `ls_vif_matrix_stability`); note that κ is the condition number of Z, i.e. the square root of that of $Z^\top Z$. These results concern the linearised problem; they do not establish identifiability of the nonlinear model. Step 9 proves that the greedy selection returns a subset

with $\kappa \le \tau_\kappa$ and VIF $\le \tau_{\mathrm{VIF}}$, and, because both tests are inherited by subsets, that a discarded candidate would still fail at the end (`greedy_kappa_vif_guarantee`).

Steps 10–13 (complements). Two parameters with proportional sensitivity columns ($Ze_a = \lambda Ze_b$) give $Z(e_a - \lambda e_b) = 0$, so no κ threshold accepts both and the estimate is not unique; zero discarded energy makes the reduction exact to first order (step 10). Values computed with finite differences and rounding certify R_{var}, κ and VIF for the exact matrix when explicit margins hold (step 11). The linearised e_{fit} used for large systems differs from the exact one by at most $M\rho^2$ (step 12). Error bounds transfer from states to measured outputs through Lipschitz observation functions, including log and $\log_{10}$ (step 13).

Part II. Ordinary differential equations (steps 14–18)

Part I needs F to be differentiable, with known derivative, for every θ near θ_0. Part II proves this for ODE models.

Step 14. If $v(t,\cdot)$ is K-Lipschitz, Picard iteration on intervals of length at most $1/(2K)$ gives a solution on all of $[a,b]$ (`exists_solution_Icc`).

Step 15 (sensitivities). If f is C^1 on an open set U containing the nominal trajectory and x^0 is differentiable, then $x(t;\theta)$ is differentiable in θ at θ_0 for every $t \in [0,T]$, and its derivative $S(t)$ solves the variational equation

$$\dot S(t) = \partial_x f(x_0(t),\theta_0)\, S(t) + \partial_\theta f(x_0(t),\theta_0), \qquad S(0) = \partial_\theta x^0(\theta_0)$$

(`hasFDerivAt_solution_param_init`). The proof combines a uniform linearisation of f near the compact trajectory with a Grönwall estimate in a tube around it.

Step 16. Solutions for θ near θ_0 exist automatically once the nominal solution exists (`exists_solutions_near`), so this need not be assumed.

Step 17 (explicit constants). If $\|Df\| \le M_1$ and Df is M_2-Lipschitz on a convex set containing the trajectories,

$$\|x(t;\theta) - x(t;\theta')\| \le (e^{M_1 T} - 1)\|\theta - \theta'\|, \qquad \|S_\theta(t) - S_{\theta'}(t)\| \le M_2 e^{2M_1 T}\frac{e^{M_1 T} - 1}{M_1}\|\theta - \theta'\|$$

(`traj_lipschitz`, `sens_lipschitz`).

Step 18. The same holds segment by segment for systems with events at fixed times (`event_hasFDerivAt`).

Part III. Certificates for ODE models (steps 19–22)

Part III applies Part I to $F = \mathcal{F}$, the vector of states at the sampling times, whose derivative is the sampled sensitivity matrix $\mathcal{S} = (S(t_1), \dots, S(t_N))$ given by Part II.

Step 19. The energy criterion computed from $\mathcal{S}$ bounds the trajectory error of the reduced model: for every $\eta > 0$ and small $\Delta\theta$,

$$R_{\mathrm{var}}(S) \ge r \;\Rightarrow\; \|\mathcal{F}(\theta_0 + \Delta\theta) - \mathcal{F}(\theta_0 + \Delta\theta_S)\| \le \big(\sqrt{(1-r)E} + \eta\big)\|\Delta\theta\|$$

(`certified_parameter_reduction`).

Step 20. Step 17, applied at the N sampling times, provides the Lipschitz constant needed in step 4, $L_c = \sqrt{N}\, M_2 e^{2M_1 T}(e^{M_1 T} - 1)/M_1$, so the finite-perturbation certificate of $\cos\Delta$ becomes fully explicit for ODE models (`certified_cos_ode`).

Step 21. The certificate is sharpened by using the exact linear error $\|\mathcal{S}(h - h_S)\|$ instead of its energy bound, and a local version without any Lipschitz constant is proved: $\lim_{s\to 0^+} \cos\Delta(sh) = \cos(\mathcal{S}h, \mathcal{S}h_S)$, so if the linear cosine computed from $\mathcal{S}$ exceeds c, then $\cos\Delta(sh) > c$ for all small $s > 0$ (`certified_cos_ode_exact`, `local_cos_ode`).

Step 22. Combining steps 16 and 21, the local certificate requires only that the nominal solution exist (`local_cos_from_nominal`).

Part IV. Hypotheses of the benchmark models (steps 23–31)

The results of Parts II–III assume that f is C^1 on an open set containing the trajectory and that the nominal solution exists on $[0, T]$. Part IV proves these assumptions for the benchmark models from their equations.

Step 23 (smoothness). Rate laws are represented as expressions built from constants, concentrations, parameters, $+, -, \times, \div$, powers, exp and log. Wherever an expression is well defined (non-zero denominators, positive bases of real powers and arguments of log) it is C^∞, and this set is open (`contDiffOn_field`); Michaelis–Menten and Hill laws are covered (`ok_mm`, `ok_hill`, `ok_hillR`). This yields the local certificate for every kinetic model (`kinetic_local_cos`).

Step 24 (positivity). If f is quasi-positive ($y \geq 0$, $y_i = 0 \Rightarrow f_i(y) \geq 0$), solutions starting in the non-negative orthant stay there (`nonneg_of_quasiPositive`); the proof applies Grönwall's inequality to the squared negative part.

Step 25 (global existence). If, in addition, $\sum_i w_i f_i(y) \leq a + b\sum_i w_i y_i$ on the orthant for weights $w_i > 0$, then $\sum_i w_i x_i(t)$ grows at most exponentially, the solution remains in a box $[0, B]^n$ on which f is Lipschitz, and by step 14 it exists on $[0, T]$ for every T (`exists_global_solution`).

Step 26. The same holds when some species must remain strictly positive (for example, when they appear in denominators), provided their consumption is at most proportional to their concentration (`exists_global_solution_strict`).

Steps 27–28 (automatic checks). Boolean functions that inspect the syntax of the rate laws decide well-definedness, sign and quasi-positivity, and each is proved sound (for example `qp_sound`). For a reaction network $f_i = \sum_k \nu_{ik} r_k$, the checks `checkNet` and `checkGrowth` imply the hypotheses of steps 23–25 (`checkNet_sound`, `checkGrowth_sound`), and the end-to-end theorem `network_final` gives existence, positivity and differentiability for every horizon.

Steps 29–31. Three extensions cover the remaining models: quadratic growth, $\sum_i w_i f_i \leq Q(\sum_i w_i y_i + 1)^2$, gives existence up to the Riccati time $1/(Q(\sum_i w_i x_i^0 + 1))$ (`riccati_final`, used for Crauste with $T \leq 1$); networks with strictly positive domains (`strict_final_init`); and parameter-dependent initial values checked with rational interval arithmetic (`strict_final_initI`).

Benchmark models. Each file in `lean_proofs/Models/` contains the rate laws, stoichiometry, parameter values and initial values of one model, translated from SBML with exact rational numbers. The checks of steps 27–31 are executed by the Lean kernel (`decide +kernel`), and one end-to-end theorem is then applied: `network_final` (15 models), `network_final_init` (5), `network_segments_final` (1), `strict_final_init` (5), `strict_final_initI` (1) or `riccati_final` (1). The resulting theorem states that the model satisfies the hypotheses of Parts II–III. Supplementary Note 4 gives the data and the checks of each model.

Connection with the numerical results and what the proofs do not cover

Lean proves implications; it does not recompute the numerical results. The sensitivity matrices, R_{var}, κ, VIF, $\cos\Delta$ and e_{fit} reported in the paper are computed in Python, in double precision. Three levels of evidence should therefore be distinguished.

- *General mathematics (G1–G4).* Proved in Lean for every model that satisfies the stated hypotheses.
- *Hypotheses of specific models (G5).* Proved in Lean for 28 benchmark models from their translated equations; the translation was compared independently with the SBML models (Supplementary Note 4).
- *Numerical conditions.* The inequalities required by the theorems ($\kappa \leq 10$, VIF ≤ 10, median $e_{\text{fit}} \leq 0.436$, and $R_{\text{var}} \geq r$ for the bound of G1) were evaluated in floating-point arithmetic; Supplementary Table S4 gives their values. The certified margins of step 11 were not applied to these computations. In a preliminary analysis with an earlier version of the selection, the explicit finite-perturbation certificates of steps 4 and 20 were evaluated with bounds M_1, M_2 computed from the vector fields: the resulting constants L_c ranged from about 10^2 to 10^{24} and no scenario at $\pm 5\%$ could be certified in any model (`certificados/resultados/tabla_certificados.md`). These certificates are therefore of theoretical value; in practice the method relies on the local results (steps 19, 21 and 22) and on the numerical admissibility test. The conclusions of G1–G3 for a particular problem are therefore conditional on the computed values; for the closest cases (VIF = 9.85 in Lucarelli and $e_{\text{fit}} = 0.4355$ in Lucarelli) a small numerical error could change the decision.

The sensitivities used by the method are relative, $J = \partial y / \partial \log\theta$; the theorems apply to them through the change of variables $\theta = e^u$, which is smooth for $\theta > 0$. Finally, the formal results are local, around the published estimate θ_0; the behaviour for large parameter changes is assessed empirically in the paper.

Supplementary Note 3. Statements of the formal theorems

This note states, in mathematical notation, every theorem and lemma of the Lean formalisation (31 modules, 289 declarations) and explains what it means, so that the formal statements can be read alongside the code. The Lean name is given as it appears in the source (prefixed by the module namespace). Hypotheses are written before $\Rightarrow$; standing assumptions of each module are given in its description. Table S5 explains the symbols used throughout; symbols specific to a module are explained at the start of that module. The same content, with the Lean identifiers, is in `lean_proofs/FORMULAS.md`.

Supplementary Table S5. Notation of the formalisation.

Symbol	Lean	Meaning
$n,\ p,\ N$	`n, p, N`	number of state variables, of parameters and of sampling times
$\theta \in \mathbb{R}^p,\ \theta_0$	θ, θ_0	parameter vector; nominal (published) parameter values
$\dot{x} = f(x,\theta)$	`f`	right-hand side of the ODE (vector field)
$x(t;\theta) \in \mathbb{R}^n$	`x θ t`	solution of the ODE at time t for parameters θ
$x_0(t)$	$\mathtt{x}_0$	nominal solution, i.e. the solution for $\theta = \theta_0$

Supplementary Table S5. Notation of the formalisation.

Symbol	Lean	Meaning
$x^0(\theta),\ D_0$	`x0, D0`	initial value as a function of θ, and its derivative at θ_0
U	`U`	open set of (state, parameter) pairs on which f is continuously differentiable (ContDiffOn $\mathbb{R}$ 1)
K	`K`	convex set containing all trajectories considered (used for explicit constants)
$[0, T]$	`Icc 0 T`	simulation horizon
x solves the ODE	`HasDerivWithinAt (x θ) (f (x θ t) θ) (Icc 0 T) t`	x(·;θ) is differentiable on [0,T] and its derivative equals f(x,θ)
$t_1, \ldots, t_N$	`tk`	sampling (measurement) times
$\mathcal{F}(\theta) \in \mathbb{R}^{Nn}$	`sampled x tk θ`	states at all sampling times stacked into one vector
$F : \mathbb{R}^p \to \mathbb{R}^m$	`F`	generic differentiable output map (for example the stacked, noise-scaled measured outputs)
$DF(\theta),\ J$	`DF θ, J, L`	derivative of F (sensitivity matrix); J = DF(θ_0)
J_{ij}	`jacobianOf L`	entry (i, j) of the matrix of the linear map L: J_ij = (L e_j)_i
$S(t) = \partial x(t;\theta_0)/\partial\theta$	`S, S₀`	state sensitivity matrix; solves the variational equation
$\mathcal{S}$	`sampledCLM S tk, sensMatrix S tk`	sensitivity at all sampling times stacked into one Nn × p matrix
$\dot{S} = \partial_x f\, S + \partial_\theta f$	`HasDerivWithinAt S (... ∘L S t + ...)`	variational (sensitivity) equation along the nominal solution
$S \subseteq \{1, \ldots, p\}$	`S, Sset : Finset (Fin p)`	subset of parameters that are re-estimated; the others stay at θ_0
S^c	`removedSet S`	discarded (fixed) parameters
$h,\ \Delta\theta$	`h, Δθ`	parameter perturbation
h_S	`keepS S h`	perturbation restricted to S (components outside S set to 0)
$E_j = \sum_i J_{ij}^2$	`columnEnergy J j`	sensitivity energy of parameter j (diagonal entry of $J^\top J$)
$E = \mathrm{tr}(J^\top J)$	`totalColumnEnergy J`	total sensitivity energy
$E_S,\ E_{S^c}$	`keptColumnEnergy J S, removedColumnEnergy J S`	energy of the selected and of the discarded parameters
$R_{\mathrm{var}}(S) = E_S/E$	`r * totalColumnEnergy ≤ keptColumnEnergy`	fraction of sensitivity energy kept; the condition R_var(S) ≥ r is written r·E ≤ E_S
$\|h_{S^c}\|^2 = \sum_{j\notin S} h_j^2$	`removedParamSq S h`	squared size of the perturbation of the discarded parameters
$\cos(a,b) = \dfrac{\langle a,b\rangle}{\|a\|\,\|b\|}$	`cosv a b`	cosine between two vectors
$\cos\Delta(h)$	`cosv (F (θ₀+h) - F θ₀) (F (θ₀+h_S) - F θ₀)`	cosine between the output change of the full perturbation and that of the perturbation restricted to S
c	`c`	cosine threshold (the method uses c = 0.9, i.e. $\tau_e = \sqrt{(1 - c^2)} = 0.436$)
$\bar{B}(\theta_0, \rho_0)$	`closedBall θ₀ ρ₀`	closed ball of radius ρ_0 around θ_0 (range of perturbations)
L_c	`Lc`	Lipschitz constant of DF on the ball: ‖DF(θ) – DF(θ_0)‖ ≤ L_c ‖θ – θ_0‖
$M_1,\ M_2$	M_1, M_2	bounds on ‖Df‖ and on the Lipschitz constant of Df on K

Supplementary Table S5. Notation of the formalisation.

Symbol	Lean	Meaning
Z	`Z`	linear map whose columns are the (normalised) sensitivity columns of the candidate subset
$\sigma_{\min},\ \sigma_{\max}$	`σmin, σmax`	lower and upper bounds $\sigma_{\min}\|u\| \le \|Zu\| \le \sigma_{\max}\|u\|$ (extreme singular values)
$\kappa_0,\ V$	`κ₀, V, Vb`	thresholds on the condition number κ and on the variance inflation factor VIF
$\operatorname{supp} u \subseteq S$	`SupportedOn S u`	u_i = 0 for all i outside S
$v \in \mathrm{LS}(Z, y)$	`IsLSMin Z y v`	v minimises ‖Zv − y‖ (least-squares estimate)
$\forall \theta \approx \theta_0$	`∀ᶠ θ in 𝓝 θ₀`	for all θ in some neighbourhood of θ_0
$o(\|\Delta\theta\|)$	`=o[𝓝 0]`	little-o as $\Delta\theta \to 0$
$e,\ e(x, \theta)$	`e : KExpr n p, e.eval (x, θ)`	syntax tree of a rate law built from constants, x_i, θ_j, +, −, ×, ÷, integer and real powers, exp and log; its value
$\mathrm{ok}_e(x, \theta)$	`e.ok (x, θ)`	e is well defined at (x, θ): non-zero denominators, positive bases of real powers and positive arguments of log
$F_1, \dots, F_n;\ f_F$	`F : Fin n → KExpr n p, field F x θ`	rate expressions of a kinetic model and the vector field they define
$\mathcal{D}_F$	`domain F`	set of (x, θ) where all F_i are well defined
$P,\ \theta_P > 0$	`pos, PosParams pos θ`	parameters declared positive, and the condition θ_j > 0 for those parameters
$y \ge 0$	`Nonneg y`	all components of y are non-negative
$\Sigma,\ y \in P_\Sigma$	`sx, SPos sx y`	species required to stay strictly positive; y ≥ 0 and y_j > 0 for j ∈ Σ
(r_k, ν_k)	`Rx : List (KExpr × List (Fin n × ℚ))`	reaction k with rate law r_k and stoichiometric column ν_k
$f_i = \sum_k \nu_{ik} r_k$	`netF Rx, coef`	vector field of a reaction network; ν_ik = coef
w_i	`c : Fin n → ℚ`	positive weights of the growth (Lyapunov-type) bound Σ_i w_i f_i ≤ a + b Σ_i w_i x_i
$\theta_q,\ x_q$	`θq, xq`	rational parameter values and initial values taken from the model files
$\iota(q)$	`qvec q`	rational vector viewed as a real vector
`check`$(\cdot) = \text{true}$	`checkModel, checkNet, ... = true`	a Boolean checker, executed by Lean on the model, returns true

Part I. Finite-dimensional core of the method (fixed sensitivity matrix)

Step 1. `FirstOrderErrorBound` (12)

First-order (linearised) output error when the parameters outside S are fixed. Here J is a numeric m × p matrix.

Notation: $(J\Delta\theta)_i$ (`linResponse J Δθ i`): linear response of output i: Σ_j J_ij Δθ_j; $(J\Delta\theta_S)_i$ (`reducedResponse J S Δθ i`): linear response with only the parameters in S perturbed; $\|v\|^2$ (`sqNorm v`): sum of squares of the components of v.

Lean name	Statement	Meaning
`sqNorm_nonneg`	$\|v\|^2 \geq 0$	A sum of squares is non-negative.
`columnEnergy_nonneg`	$E_j \geq 0$	Each sensitivity energy is non-negative.
`removedColumnEnergy_nonneg`	$E_{S^c} \geq 0$	The discarded energy is non-negative.
`removedParamSq_nonneg`	$\|\Delta\theta_{S^c}\|^2 \geq 0$	The size of the discarded perturbation is non-negative.
`sum_split`	$\sum_j f_j = \sum_{j\in S} f_j + \sum_{j\notin S} f_j$	A sum over all parameters splits into selected plus discarded ones.
`total_eq_kept_add_removed`	$\mathrm{tr}(J^\top J) = E_S + E_{S^c}$	Total energy = kept energy + discarded energy.
`response_sub_reduced`	$(J\Delta\theta)_i - (J\Delta\theta_S)_i = \sum_{j\notin S} J_{ij}\Delta\theta_j$	The truncation error is exactly the contribution of the discarded columns.
`first_order_error_sq_bound`	$\|J\Delta\theta - J\Delta\theta_S\|^2 \leq E_{S^c}\,\|\Delta\theta_{S^c}\|^2$	First-order error bound (squared form, Cauchy–Schwarz).
`first_order_error_bound`	$\|J\Delta\theta - J\Delta\theta_S\| \leq \sqrt{E_{S^c}}\,\|\Delta\theta_{S^c}\|$	First-order error bound (norm form).
`removedParamSq_le`	$\|\Delta\theta_{S^c}\|^2 \leq \|\Delta\theta\|^2$	The discarded part of a perturbation is no larger than the whole perturbation.
`first_order_error_from_removed_fraction`	$E_{S^c} \leq \varepsilon E \;\Rightarrow\; \|J\Delta\theta - J\Delta\theta_S\|^2 \leq \varepsilon\, E\,\|\Delta\theta\|^2$	If at most a fraction ε of the energy is discarded, the first-order error is at most $\sqrt{(\varepsilon \mathrm{E})}\|\Delta\theta\|$.
`first_order_error_from_Rvar`	$rE \leq E_S \;\Rightarrow\; \|J\Delta\theta - J\Delta\theta_S\|^2 \leq (1-r)\,E\,\|\Delta\theta\|^2$	Stage-1 criterion: R_var(S) $\geq$ r bounds the first-order error by $\sqrt{((1 - \mathrm{r})\mathrm{E})}\|\Delta\theta\|$.

Step 2. `TrajectoryErrorBound` **(9)**

From the linear map to the true (nonlinear) output: error of the reduced model for small perturbations.

Lean name	Statement	Meaning
`keepS_apply`	$(h_S)_j = \begin{cases} h_j & j \in S \\ 0 & j \notin S \end{cases}$	Definition of the restricted perturbation.
`norm_keepS_le`	$\|h_S\| \leq \|h\|$	Restricting a perturbation cannot increase its size.
`clm_apply_eq_linResponse`	$(L\Delta\theta)_i = \sum_j J_{ij}\Delta\theta_j$	A linear map acts through its matrix J.
`clm_keepS_eq_reducedResponse`	$(L\Delta\theta_S)_i = \sum_{j\in S} J_{ij}\Delta\theta_j$	Same for the restricted perturbation.
`linear_part_bound`	$\|L\Delta\theta - L\Delta\theta_S\| \leq \sqrt{E_{S^c}}\,\|\Delta\theta_{S^c}\|$	Linear error bound for a linear map.
`sqrt_removedParamSq_le`	$\|\Delta\theta_{S^c}\| \leq \|\Delta\theta\|$	Norm form of the previous inequality on perturbations.
`remainder_isLittleO`	$\big[F(\theta_0{+}\Delta\theta) - F(\theta_0{+}\Delta\theta_S)\big] - \big[L\Delta\theta - L\Delta\theta_S\big] = o(\|\Delta\theta\|)$	If F is differentiable at θ_0 with derivative L, the nonlinear error differs from the linear one by a term of higher order.

Lean name	Statement	Meaning
`trajectory_error_bound`	$\forall \eta > 0,\ \forall \Delta\theta \approx 0 :\ \|F(\theta_0{+}\Delta\theta) - F(\theta_0{+}\Delta\theta_S)\| \le \sqrt{E_{S^c}}\,\|\Delta\theta_{S^c}\| + \eta\|\Delta\theta\|$	Local error bound of the reduced model for the true output map.
`trajectory_error_from_Rvar`	$rE \le E_S \Rightarrow \forall \eta > 0, \forall \Delta\theta \approx 0 :$ $\|F(\theta_0{+}\Delta\theta) - F(\theta_0{+}\Delta\theta_S)\| \le \big(\sqrt{(1-r)E} + \eta\big)\|\Delta\theta\|$	Same bound expressed with R_var(S) ≥ r.

Step 3. `ScaledError` **(5)**

Theorem 1 of the paper: the minimum relative error after rescaling the reduced response equals $\sqrt{(1 - \cos^2\Delta)}$.

Lean name	Statement	Meaning
`norm_sub_smul_sq`	$\|a - \alpha b\|^2 = \|a\|^2 - 2\alpha\langle a, b\rangle + \alpha^2\|b\|^2$	Expansion of the squared error.
`scaled_error_ge`	$\|a\|^2 - \dfrac{\langle a, b\rangle^2}{\|b\|^2} \le \|a - \alpha b\|^2 \quad \forall \alpha$	Lower bound for every scaling α (b ≠ 0).
`scaled_error_eq`	$\left\|a - \dfrac{\langle a, b\rangle}{\|b\|^2} b\right\|^2 = \|a\|^2 - \dfrac{\langle a, b\rangle^2}{\|b\|^2}$	The bound is attained at $\alpha^* = \langle$a,b$\rangle/\|$b$\|^2$.
`scaled_error_cos`	$\|a\|^2 - \dfrac{\langle a, b\rangle^2}{\|b\|^2} = \|a\|^2\big(1 - \cos^2(a, b)\big)$	Rewriting in terms of the cosine.
`min_rel_error_eq`	$\min\limits_{\alpha} \dfrac{\|a - \alpha b\|}{\|a\|} = \sqrt{1 - \cos^2(a, b)}$	Theorem 1: with a the full response and b the reduced one, the best relative error is $\sqrt{(1 - \cos^2\Delta)}$; hence cos Δ ≥ c ⇔ error ≤ τ_e.

Step 4. `CosineCertificate` **(5)**

Non-asymptotic bounds for finite perturbations and a per-scenario certificate of cos Δ.

Lean name	Statement	Meaning
`cos_lower_of_rel_error`	$c^2 + \varepsilon^2 \le 1,\ \|a - b\| \le \varepsilon\|a\| \Rightarrow c\,\|a\|\,\|b\| \le \langle a, b\rangle$	A small relative error implies a large cosine.
`cos_ge_of_rel_error`	$0 \le \varepsilon < 1,\ c \ge 0,\ c^2 + \varepsilon^2 \le 1,\ \|a - b\| \le \varepsilon\|a\|,\ a \ne 0 \Rightarrow b \ne 0,\ \cos(a, b) \ge c$	Same, with the cosine itself.
`taylor_bound`	$\|F(\theta_0 + h) - F(\theta_0) - DF(\theta_0)h\| \le L_c\|h\|^2 \quad (\|h\| \le \rho_0)$	Taylor bound when DF is L_c-Lipschitz on the ball.
`finite_trajectory_error`	$\|F(\theta_0 + h) - F(\theta_0 + h_S)\| \le \sqrt{E_{S^c}}\,\|h_{S^c}\| + 2L_c\|h\|^2$	Error of the reduced model for a finite perturbation (E computed from J = DF(θ_0)).
`finite_cos_certificate`	$A = \|Jh\| - L_c\|h\|^2 > 0,\ \big(\sqrt{E_{S^c}}\|h_{S^c}\| + 2L_c\|h\|^2\big)^2 \le (1 - c^2)A^2 \Rightarrow \cos\Delta(h) \ge c$	Certificate of cos Δ for one perturbation h, computable from J, L_c and ‖h‖.

Step 5. `RefitMonotone` **(5)**

Re-estimating more parameters never increases the fitting error.

Notation: $\mathcal{B}$ (`B`): admissible parameter region (bounds); $r(\theta) \geq 0$ (`r`): fitting error of θ (for example the relative output error); feas(S) (`feas θ₀ B S`): $\{\theta \in \mathcal{B} : \theta_\mathtt{j} = \theta_{0,j} \text{ for } \mathtt{j} \notin \mathrm{S}\}$; $e_{\mathrm{fit}}(S) = \inf_{\theta \in \mathrm{feas}(S)} r(\theta)$ (`fitErr r θ₀ B S`): best error when only S is re-estimated.

Lean name	Statement	Meaning
`feas_mono`	$S \subseteq T \Rightarrow \mathrm{feas}(S) \subseteq \mathrm{feas}(T)$	More free parameters, more candidates.
`fitErr_anti`	$S \subseteq T \Rightarrow e_{\mathrm{fit}}(T) \leq e_{\mathrm{fit}}(S)$	The fitting error is anti-monotone in S.
`fitErr_union_le`	$e_{\mathrm{fit}}(S \cup C) \leq e_{\mathrm{fit}}(S)$	Adding the calibration parameters C never hurts.
`fitErr_le_erel`	$\theta_s \in \mathrm{feas}(S) \Rightarrow e_{\mathrm{fit}}(S) \leq r(\theta_s)$	e_fit ≤ e_rel: re-estimating is never worse than keeping the perturbed values of S.
`admissible_mono`	$S \subseteq T,\ e_{\mathrm{fit}}(S) \leq \tau \Rightarrow e_{\mathrm{fit}}(T) \leq \tau$	Admissibility is preserved when parameters are added.

Step 6. `Poda` (2)

Theorem 2 of the paper: backward elimination (pruning) returns an inclusion-minimal admissible subset.

Notation: $Q(S)$ (`Q`): admissibility predicate; prune(S) (`prune Q pick hpick S`): repeatedly remove an element whose removal keeps Q, until none exists.

Lean name	Statement	Meaning
`prune_spec`	$Q(S) \Rightarrow Q(R) \ \wedge\ R \subseteq S \ \wedge\ \forall x \in R : \neg Q(R \setminus \{x\}), \quad R = \mathrm{prune}(S)$	Pruning keeps admissibility and stops at a set from which no single parameter can be removed.
`exists_minimal_admissible`	$Q(S) \Rightarrow \exists R \subseteq S :\ Q(R) \ \wedge\ \forall x \in R : \neg Q(R \setminus \{x\})$	Every admissible subset contains an inclusion-minimal admissible subset.

Step 7. `IdentifiabilityConditioning` (10)

Bounds on κ and VIF imply unique and stable least-squares estimates.

Lean name	Statement	Meaning
`IsLSMin.inner_residual_eq_zero`	$v \in \mathrm{LS}(Z, y) \Rightarrow \langle Zv - y, Zw \rangle = 0 \ \ \forall w$	Normal equations: the residual is orthogonal to the range of Z.
`ls_response_stability`	$\|Z(v - v')\| \leq \|y - y'\|$	Fitted responses are 1-Lipschitz in the data (v, v′ least-squares estimates for data y, y′).
`ls_param_stability`	$\sigma_{\min}\|u\| \leq \|Zu\| \Rightarrow \|v - v'\| \leq \|y - y'\|/\sigma_{\min}$	Parameter estimates are stable when $\sigma_{\min} > 0$.
`ls_unique`	$\sigma_{\min} > 0 \Rightarrow v = v'$ (same y)	The least-squares estimate is unique.
`one_le_kappa_mul_sigmaMin`	$\|Ze_j\| = 1,\ \sigma_{\max} \leq \kappa\sigma_{\min} \Rightarrow 1 \leq \kappa\,\sigma_{\min}$	With unit columns, $\sigma_{\max} \geq 1$.
`ls_kappa_stability`	$\|Ze_j\| = 1 \Rightarrow \|v - v'\| \leq \kappa\,\|y - y'\|$	With normalised columns, κ bounds the sensitivity of the estimates to the data.

Lean name	Statement	Meaning
`ls_param_kappa_stability`	$\|D(\theta-\theta')\| \le \kappa \|y-y'\|$	Same for the estimates in the original scale, $\theta = \mathrm{D}^{-1}\mathrm{v}$ with a column-scaling D.
`ls_vif_coordinate_stability`	$\big(u_j = 1 \Rightarrow \|Zu\|^2 \ge 1/V\big) \Rightarrow \|v_j - v'_j\| \le \sqrt{V}\,\|y-y'\|$	A bounded VIF makes each coordinate stable.
`vif_geometric_of_inverse`	$\langle Zw, Zu\rangle = u_j\ \forall u,\ w_j \le V \Rightarrow \big(u_j = 1 \Rightarrow \|Zu\|^2 \ge 1/V\big)$	The VIF computed from the inverse Gram matrix (w = $(\mathrm{Z}^\top\mathrm{Z})^{-1}$e_j, VIF_j = w_j) implies the geometric condition.
`ls_vif_inverse_stability`	$w_j \le V \Rightarrow \|v_j - v'_j\| \le \sqrt{V}\,\|y-y'\|$	Coordinate stability from the VIF as computed numerically.

Step 8. `SpectralConditioning` **(22)**

κ and VIF exactly as computed numerically: eigenvalues of the Gram matrix and the inverse matrix.

Notation: $G = Z^\top Z$ (`gram Z`): Gram operator; $\lambda_1 \ge \cdots \ge \lambda_k$, v_i (`lam Z, evec Z`): eigenvalues (decreasing) and orthonormal eigenvectors of G; $\lambda_{\max} = \lambda_1$, $\lambda_{\min} = \lambda_k$ (`lam Z (iMax hk), lam Z (iMin hk)`): largest and smallest eigenvalue (k > 0); $\kappa = \sqrt{\lambda_{\max}/\lambda_{\min}}$: condition number of Z; M (`M : Matrix (Fin m) (Fin k) ℝ, matCLM M`): numeric matrix and the linear map it defines.

Lean name	Statement	Meaning
`inner_gram`	$\langle Gu, v\rangle = \langle Zu, Zv\rangle$	Defining property of the Gram operator.
`gram_isSymmetric`	$G = G^\top$	The Gram operator is symmetric.
`finrank_eq`	$\dim \mathbb{R}^k = k$	Dimension (needed for the spectral theorem).
`lam_antitone`	$\lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_k$	Eigenvalues are sorted in decreasing order.
`gram_evec`	$Gv_i = \lambda_i v_i$	Eigenvector equation.
`sq_norm_apply_evec`	$\|Zv_i\|^2 = \lambda_i$	Eigenvalues are squared singular values.
`lam_nonneg`	$\lambda_i \ge 0$	The Gram matrix is positive semidefinite.
`sq_norm_apply_eq_sum`	$\|Zu\|^2 = \sum_i \lambda_i \langle v_i, u\rangle^2$	Spectral decomposition of $\|\mathrm{Zu}\|^2$.
`sq_norm_eq_sum`	$\|u\|^2 = \sum_i \langle v_i, u\rangle^2$	Parseval identity in the eigenbasis.
`sq_norm_bounds`	$lo \le \lambda_i \le hi\ \forall i \Rightarrow lo\,\|u\|^2 \le \|Zu\|^2 \le hi\,\|u\|^2$	Eigenvalue bounds give norm bounds.
`lam_le_max`	$\lambda_i \le \lambda_{\max}$	Largest eigenvalue.
`min_le_lam`	$\lambda_{\min} \le \lambda_i$	Smallest eigenvalue.
`norm_bounds`	$\sqrt{\lambda_{\min}}\,\|u\| \le \|Zu\| \le \sqrt{\lambda_{\max}}\,\|u\|$	Extreme singular values bound $\|\mathrm{Zu}\|$.
`opNorm_eq_sqrt_lamMax`	$\|Z\| = \sqrt{\lambda_{\max}}$	The operator norm is the largest singular value.
`sqrt_lamMin_mul_le`	$\sqrt{\lambda_{\min}}\,\|u\| \le \|Zu\|$	Lower bound by the smallest singular value.
`le_sqrt_lamMin`	$\big(\forall u:\ \sigma\|u\| \le \|Zu\|\big) \Rightarrow \sigma \le \sqrt{\lambda_{\min}}$	$\sqrt{\lambda_{\min}}$ is the best possible lower constant.

Lean name	Statement	Meaning
`ls_kappa_stability_spectral`	$\|Ze_j\| = 1,\ \lambda_{\min} > 0 \Rightarrow \|v - v'\| \le \sqrt{\lambda_{\max}/\lambda_{\min}}\,\|y - y'\|$	Stability of least squares with κ exactly as computed (ratio of singular values).
`matCLM_apply`	$(\mathrm{matCLM}\, M)\, u = Mu$	A matrix acts on vectors by multiplication.
`inner_matCLM`	$\langle Mu, Mv\rangle = u^\top (M^\top M) v$	Inner products through the Gram matrix.
`gram_toEuclideanLin`	$G(\mathrm{matCLM}\, M) = M^\top M$	The abstract Gram operator is the matrix $\mathrm{M}^\top\mathrm{M}$.
`vif_matrix_inverse`	$\det(M^\top M) \ne 0,\ w = (M^\top M)^{-1} e_j \Rightarrow \langle Mw, Mu\rangle = u_j\ \forall u,\ w_j = [(M^\top M)^{-1}]_{jj}$	The diagonal of the inverse Gram matrix (the VIF for normalised columns) satisfies the hypothesis of vif_geometric_of_inverse.
`ls_vif_matrix_stability`	$[(M^\top M)^{-1}]_{jj} \le V \Rightarrow \|v_j - v'_j\| \le \sqrt{V}\,\|y - y'\|$	Coordinate stability with the VIF exactly as computed numerically.

Step 9. `GreedySelection` (7)

The greedy selection algorithm with permanent discard and its guarantees.

Notation: $G = \mathrm{greedy}(\mathrm{ok}, \mathrm{stop}, l, S)$ (`greedy ok stop l S`): scan the ordered list l; add j if ok(S ∪ {j}), otherwise discard j permanently; stop when stop(S); okKappa(Z, κ_0, S) (`okKappa Z` κ_0 `S`): $\exists\, \sigma_{\min} > 0, \sigma_{\max} \le \kappa_0 \sigma_{\min}$ with $\sigma_{\min}\|\mathrm{u}\| \le \|\mathrm{Zu}\| \le \sigma_{\max}\|\mathrm{u}\|$ for all u supported on S ($\kappa \le \kappa_0$); okVIF(Z, V, S) (`okVIF Z V S`): for j ∈ S and every u supported on S with u_j = 1: $\|\mathrm{Zu}\|^2 \ge 1/\mathrm{V}$ (VIF_j ≤ V).

Lean name	Statement	Meaning
`greedy_ok`	$\mathrm{ok}(S) \Rightarrow \mathrm{ok}(G)$	The result always passes the test.
`greedy_subset`	$S \subseteq G \subseteq S \cup l$	The result only contains the start set and candidates from the list.
`greedy_discard_permanent`	ok anti-monotone, $j \in l,\ j \notin G \Rightarrow \neg\,\mathrm{ok}(G \cup \{j\})\ \vee\ \mathrm{stop}(G)$	Discarding is never a mistake: a rejected candidate would still fail with the final set.
`supportedOn_mono`	$S \subseteq T,\ \mathrm{supp}\, u \subseteq S \Rightarrow \mathrm{supp}\, u \subseteq T$	Support is monotone.
`okKappa_anti`	$S \subseteq T,\ \mathrm{okKappa}(T) \Rightarrow \mathrm{okKappa}(S)$	Removing parameters never worsens the condition number.
`okVIF_anti`	$S \subseteq T,\ \mathrm{okVIF}(T) \Rightarrow \mathrm{okVIF}(S)$	Removing parameters never worsens the VIF.
`greedy_kappa_vif_guarantee`	$R = \mathrm{greedy}(\mathrm{okKappa} \wedge \mathrm{okVIF}, \mathrm{stop}, l, \emptyset)$: $\kappa(R) \le \kappa_0,\ \mathrm{VIF}(R) \le V,\ R \subseteq l,\ \forall j \in l \setminus R:\ R \cup \{j\}$ fails $\vee$ stop(R)	Guarantee of the selection step: the returned subset satisfies $\kappa \le \tau_\kappa$ and VIF $\le \tau_{\mathrm{VIF}}$, and every discarded candidate would violate one of them.

Step 10. `PatternMechanisms` (4)

Mechanisms behind the robustness patterns: opposing effects and zero discarded energy.

Lean name	Statement	Meaning
`not_okKappa_of_kernel`	$u \neq 0,\ \operatorname{supp} u \subseteq S,\ Zu = 0 \Rightarrow \neg\,\mathrm{okKappa}(Z, \kappa_0, S)\ \forall \kappa_0$	If two or more parameters in S have exactly compensating effects, no threshold accepts S.
`ls_not_unique_of_kernel`	$Zu = 0,\ u \neq 0,\ v \in \mathrm{LS}(Z, y) \Rightarrow v + u \in \mathrm{LS}(Z, y),\ v + u \neq v$	In that case the estimate is not unique (non-identifiability).
`opposing_pair_kernel`	$Ze_a = \lambda Ze_b,\ a \neq b \Rightarrow e_a - \lambda e_b \neq 0,\ Z(e_a - \lambda e_b) = 0$	Two proportional sensitivity columns give a kernel vector.
`exact_reduction_of_zero_energy`	$E_{S^c} = 0 \Rightarrow J\Delta\theta = J\Delta\theta_S$	If the discarded parameters have zero sensitivity, the reduction is exact to first order.

Step 11. `NumericalRobustness` (6)

From approximate numerical values (finite differences, rounding) to the exact hypotheses, with explicit margins.

Notation: $\tilde{J}$, $\tilde{Z}$ (`Jt, Zt`): approximate (computed) matrices; τ, τ_j (τ): bounds on the approximation error; $J_{\cdot j}$ (`col J j`): column j of J.

Lean name	Statement	Meaning
`fd_column_error`	$\|v\| = 1,\ 0 < \delta \leq \rho_0 \Rightarrow \left\| \frac{F(\theta_0 + \delta v) - F(\theta_0)}{\delta} - DF(\theta_0)v \right\| \leq L_c\,\delta$	Error of a finite-difference column with step δ.
`columnEnergy_eq_norm_sq`	$E_j = \|J_{\cdot j}\|^2$	Energy of a column is its squared norm.
`rvar_certified_from_approx`	$\|J_{\cdot j} - \tilde{J}_{\cdot j}\| \leq \tau_j,\ r\Big(\sum_{j\in S}(\|\tilde{J}_{\cdot j}\| - \tau_j)_+^2 + \sum_{j\notin S}(\|\tilde{J}_{\cdot j}\| + \tau_j)^2\Big) \leq \sum_{j\in S}(\|\tilde{J}_{\cdot j}\| - \tau_j)_+^2 \Rightarrow rE(J) \leq E_S(J)$	R_var(S) $\geq$ r for the exact matrix follows from a check on the approximate one with margins.
`opNorm_le_frobenius`	$\|Du\| \leq \sqrt{\sum_j \|De_j\|^2}\,\|u\|$	Operator norm bounded by the Frobenius norm.
`kappa_certified_from_approx`	$\|(Z - \tilde{Z})u\| \leq \tau\|u\|,\ \tau < \tilde{\sigma}_{\min},\ \tilde{\sigma}_{\max} + \tau \leq \kappa_0(\tilde{\sigma}_{\min} - \tau) \Rightarrow \mathrm{okKappa}(Z, \kappa_0, S)$	$\kappa \leq \kappa_0$ for the exact matrix from bounds on the approximate one.
`vif_certified_from_approx`	$\mathrm{okVIF}(\tilde{Z}, \tilde{V}, S),\ \tau < \tilde{\sigma}_{\min} \Rightarrow \mathrm{okVIF}\Big(Z, \frac{\tilde{V}}{(1 - \tau/\tilde{\sigma}_{\min})^2}, S\Big)$	VIF bound for the exact matrix from the approximate one.

Step 12. `LinearizedFit` (3)

The linearised e_fit (used for large systems) and its relation to the exact e_fit.

Notation: $A(c)$, $F(c)$ (`A, F`): linear and exact responses as functions of the re-estimated parameters c; d (`d`): target response (perturbed full model).

Lean name	Statement	Meaning
`lin_le_scaled`	$b \in V,\ v = \arg\min_{w\in V} \|d - w\| \Rightarrow \|d - v\| \leq \|d\|\sqrt{1 - \cos^2(d, b)}$	The linear best fit in a subspace containing b is at least as good as rescaling b.

Lean name	Statement	Meaning
`fit_upper_of_lin`	$\|F(c) - A(c)\| \le M\|c\|^2 \Rightarrow \|F(c) - d\| \le \|A(c) - d\| + M\|c\|^2$	Upper bound: the exact error at the linear optimum exceeds the linear error by at most a second-order term.
`fit_lower_of_lin`	$\|F(c) - A(c)\| \le M\|c\|^2\ \forall c,\ e_{\text{lin}} \le \|A(c) - d\|\ \forall c,\ \|c\| \le \rho \Rightarrow e_{\text{lin}} - M\rho^2 \le \|F(c) - d\|$	Lower bound: no exact fit within radius ρ is better than e_lin – Mρ^2.

Step 13. `OutputComposition` (4)

Error bounds transfer from states to measured outputs y_i = h(g_i(x(t_i), θ))/σ_i, including log and log10 transforms.

Notation: G_i (`G i`): observation function of output i.

Lean name	Statement	Meaning
`output_sq_bound`	$\|G_i(x_i) - G_i(x_i')\| \le K\|x_i - x_i'\| \Rightarrow \sum_i \big(G_i(x_i) - G_i(x_i')\big)^2 \le K^2 \sum_i \|x_i - x_i'\|^2$	Lipschitz observables transfer squared state errors to outputs.
`output_uniform_bound`	$\|x_i - x_i'\| \le \varepsilon \Rightarrow \sum_{i \in s} \big(G_i(x_i) - G_i(x_i')\big)^2 \le \|s\|\, K^2 \varepsilon^2$	Uniform version.
`log_lipschitz`	$a, b \ge c > 0 \Rightarrow \|\log a - \log b\| \le \|a - b\|/c$	Natural logarithm is Lipschitz away from 0.
`log10_lipschitz`	$a, b \ge c > 0 \Rightarrow \|\log_{10} a - \log_{10} b\| \le \dfrac{\|a - b\|}{c \ln 10}$	Same for log10 (PEtab observable transformation).

Part II. Ordinary differential equations: existence and sensitivities

Step 14. `GlobalLipschitzODE` (3)

Existence of solutions on a whole interval for vector fields that are Lipschitz in the state.

Notation: $\mathrm{Sol}_{[a,c]}(\alpha)$ (`IsSolOn v a c α`): α solves α'(t) = v(t, α(t)) on [a, c].

Lean name	Statement	Meaning
`isSolOn_self`	$\mathrm{Sol}_{[a,a]}(\alpha)$	Every function solves the equation on a single point (base case).
`extend_solution`	$K(d - c) \le \frac{1}{2},\ \mathrm{Sol}_{[a,c]}(\alpha) \Rightarrow \exists \gamma :\ \gamma(a) = \alpha(a),\ \mathrm{Sol}_{[a,d]}(\gamma)$	If v(t,·) is K-Lipschitz and continuous in t, a solution can be extended by a step of length at most 1/(2K) (Picard iteration).
`exists_solution_Icc`	$\forall y_0\ \exists \alpha :\ \alpha(a) = y_0,\ \mathrm{Sol}_{[a,b]}(\alpha)$	Global existence on [a, b] for a vector field that is K-Lipschitz in the state.

Step 15. `ParamDiffODE` (8)

Differentiable dependence of ODE solutions on parameters and initial values (the derivative solves the variational equation).

Notation: ι_1, ι_2 (`ContinuousLinearMap.inl / inr`): embeddings u ↦ (u, 0) and k ↦ (0, k) of the state and parameter spaces into their product.

Lean name	Statement	Meaning
`uniform_linearization`	$\exists \delta \in (0, R]\ \forall z \in K_0,\ \|w - z\| \le \delta :$ $\|g(w) - g(z) - Dg(z)(w - z)\| \le \eta \|w - z\|$	A C^1 function is uniformly linearisable near a compact set K_0 whose R-neighbourhood lies in U, for every $\eta > 0$.
`tube_gronwall`	$\|x_\theta(t) - y_h(t)\| \le$ $\delta_0 e^{KT} + \varepsilon_g \dfrac{e^{KT} - 1}{K} \quad \forall t \in [0, T]$	Grönwall inequality in a tube: an approximate solution y_h with defect $\le \varepsilon_g$ that stays within σ of the nominal trajectory, compared with an exact solution x_θ starting δ_0 away; the exact solution does not leave the tube where v is K-Lipschitz.
`norm_comp_inl_le`	$\|L \circ \iota_1\| \le \|L\|$	Restricting a linear map on a product to the first factor does not increase its norm.
`clm_prod_apply_split`	$L(u, k) = L(\iota_1 u) + L(\iota_2 k)$	A linear map on a product splits into its state and parameter parts.
`lipschitzOn_slice`	$\|Df\| \le M$ on $\bar{B}(c, \rho) \times \{\theta\} \Rightarrow y \mapsto$ $f(y, \theta)$ is M-Lipschitz on $\bar{B}(c, \rho)$	Lipschitz constant of a slice from a bound on the derivative.
`hasFDerivAt_solution_param_init`	$\exists S :\ S(0) = D_0,\ \dot{S}(t) =$ $\partial_x f(x_0(t), \theta_0) S(t) +$ $\partial_\theta f(x_0(t), \theta_0),\ \left.\dfrac{\partial x(t;\theta)}{\partial \theta}\right\|_{\theta_0} = S(t)$	Main theorem: if f is C^1 on an open U containing the nominal trajectory, solutions exist for θ near θ_0 and x(0;θ) = $x^0(\theta)$ is differentiable, then x(t;θ) is differentiable in θ for every t ∈ [0,T] and its derivative solves the variational equation.
`hasFDerivAt_solution_param_on`	$x(0;\theta) = x(0;\theta_0) \Rightarrow \exists S :\ S(0) = 0,\ \dot{S} =$ $\partial_x f\, S + \partial_\theta f,\ \partial_\theta x(t;\theta_0) = S(t)$	Same with an initial value that does not depend on θ.
`hasFDerivAt_solution_param`	$f \in C^1(\mathbb{R}^n \times \mathbb{R}^p) \Rightarrow \exists S :\ S(0) = 0,\ \dot{S} =$ $\partial_x f\, S + \partial_\theta f,\ \partial_\theta x(t;\theta_0) = S(t)$	Same for a vector field that is C^1 everywhere.

Step 16. `LocalExistence` (9)

Solutions exist for all parameters near θ_0 as soon as the nominal solution exists (removes the hypothesis that perturbed solutions exist).

Notation: $\mathrm{clamp}_c(y)_i = \max(-c, \min(c, y_i))$ (`clampE c y`): truncation of each component to [–c, c].

Lean name	Statement	Meaning
`clampE_apply`	$\mathrm{clamp}_c(y)_i = \max(-c, \min(c, y_i))$	Definition.
`abs_clamp_sub_le`	$\|\mathrm{clamp}_c(a) - \mathrm{clamp}_c(b)\| \le \|a - b\|$	Scalar truncation is 1-Lipschitz.
`dist_clampE_le`	$\|\mathrm{clamp}_c(y) - \mathrm{clamp}_c(y')\| \le \|y - y'\|$	Vector truncation is 1-Lipschitz.
`lipschitz_clampE`	clamp_c is 1-Lipschitz	Same, as a Lipschitz statement.
`clampE_eq`	$\|y\| \le c \Rightarrow \mathrm{clamp}_c(y) = y$	Truncation does not change small vectors.
`clampE_zero`	$c \ge 0 \Rightarrow \mathrm{clamp}_c(0) = 0$	Truncation fixes the origin.
`norm_clampE_le`	$\|\mathrm{clamp}_c(y)\| \le (n + 1)\, c$	Truncated vectors are bounded.

Lean name	Statement	Meaning
`exists_solutions_near`	$\exists x(\cdot;\theta):\ x(\cdot;\theta_0)=x_0,\ \forall\theta\approx\theta_0:\ x(0;\theta)=x^0(\theta),\ \dot{x}=f(x,\theta),\ (x(t;\theta),\theta)\in U$	If f is C^1 on U, the nominal solution exists on [0,T] inside U and x^0 is continuous at θ_0, then solutions exist on the same interval for all θ close to θ_0.
`hasFDerivAt_of_nominal_solution`	$\exists x(\cdot;\theta),\ \exists S:\ S(0)=D_0,\ \dot{S}=\partial_x f\,S+\partial_\theta f,\ \partial_\theta x(t;\theta_0)=S(t)$	Existence near θ_0 and differentiability, assuming only the nominal solution.

Step 17. `SensitivityLipschitz` (8)

Explicit constants: Lipschitz dependence of trajectories and sensitivities on θ (Grönwall).

Notation: $g(z)=f(x,\theta),\ Dg$ (`g, Dg`): vector field as a function of z = (x, θ) and its derivative.

Lean name	Statement	Meaning
`gronwall_zero_init`	$u(0)=0,\ \|u'\|\le K\|u\|+\varepsilon\Rightarrow\|u(t)\|\le\varepsilon\dfrac{e^{KT}-1}{K}$	Grönwall inequality with zero initial value.
`norm_comp_inl_le`	$\|\partial_x g\|\le\|Dg\|$	The state part of a derivative is bounded by the whole derivative.
`norm_comp_inr_le`	$\|\partial_\theta g\|\le\|Dg\|$	Same for the parameter part.
`g_lipschitz`	$\|Dg\|\le M_1$ on convex $K\Rightarrow\|g(z)-g(z')\|\le M_1\|z-z'\|$	Mean-value inequality.
`norm_pair_sub_le`	$\|(a,b)-(a',b')\|\le\|a-a'\|+\|b-b'\|$	Norm on the product space.
`traj_lipschitz`	$\|x(t;\theta)-x(t;\theta')\|\le\|\theta-\theta'\|\,(e^{M_1T}-1)$	Trajectories depend Lipschitz-continuously on θ.
`sens_bound`	$\|S(t)\|\le e^{M_1T}-1$	Explicit bound on the sensitivity.
`sens_lipschitz`	$\|S_\theta(t)-S_{\theta'}(t)\|\le\|\theta-\theta'\|\,M_2\,e^{2M_1T}\dfrac{e^{M_1T}-1}{M_1}$	Sensitivities depend Lipschitz-continuously on θ; this gives the constant L_c used in the certificates.

Step 18. `EventSystems` (1)

Hypothesis 2: systems with events at fixed times (piecewise ODEs with reset maps).

Notation: $f_k,\ L_k,\ R_k$ (`f k, L k, R k`): vector field, length and reset map of segment k: y_{k+1}(0) = R_k(y_k(L_k), θ).

Lean name	Statement	Meaning
`event_hasFDerivAt`	$\exists Y_k(\cdot;\theta):\ Y_k(\cdot;\theta_0)=y_k,\ \dot{Y}_k=f_k(Y_k,\theta),\ Y_{k+1}(0;\theta)=R_k(Y_k(L_k;\theta),\theta),\ \theta\mapsto Y_k(s;\theta)$ differentiable at θ_0	If each f_k is C^1, each R_k is differentiable and the nominal segments exist, solutions exist near θ_0 on every segment and are differentiable in θ.

Part III. Certificates for ODE models

Step 19. `CertifiedODEReduction` (1)

Main local theorem for ODE models: the sensitivity exists and R_var(S) bounds the error of the reduced model.

Lean name	Statement	Meaning
`certified_parameter_reduction`	$\exists S(\cdot):\ S(0)=D_0,\ \dot S = \partial_x f\, S + \partial_\theta f,$ and $rE(\mathcal{S}) \le E_S(\mathcal{S}) \Rightarrow \forall \eta > 0, \forall \Delta\theta \approx 0:\ \|\mathcal{F}(\theta_0+\Delta\theta) - \mathcal{F}(\theta_0+\Delta\theta_S)\| \le \big(\sqrt{(1-r)E}+\eta\big)\|\Delta\theta\|$	Hypotheses: f is C^1 on an open set U containing the nominal trajectory, solutions exist for θ near θ_0 with initial value $x^0(\theta)$ differentiable at θ_0. Conclusion: the sampled sensitivity matrix $\mathcal{S}$ exists, solves the variational equation, and the energy criterion computed from it bounds the trajectory error of fixing the parameters outside S.

Step 20. `CertifiedFiniteODE` (5)

Certificate of cos Δ for ODE models with a Lipschitz constant L_c that is itself bounded formally.

Notation: $\mathcal{S}u$ (`sampledCLM S tk u`): (S(t_1)u, …, S(t_N)u), the sampled sensitivity applied to u.

Lean name	Statement	Meaning
`sampledCLM_apply`	$(\mathcal{S}u)_{(k,i)} = (S(t_k)u)_i$	Component (k, i) of the sampled sensitivity is component i of S(t_k)u.
`hasFDerivAt_sampled`	$\forall k:\ \partial_\theta x(t_k;\theta) = S(t_k) \Rightarrow D\mathcal{F}(\theta) = \mathcal{S}$	If every sampled state is differentiable, so is the stacked vector, with the stacked sensitivity as derivative.
`jacobianOf_sampledCLM`	$J(\mathcal{S}) = \texttt{sensMatrix}(S,t)$	The matrix of the sampled map is the Nn × p sensitivity matrix.
`sampledCLM_sub_le`	$\|S(t_k) - S'(t_k)\| \le B\ \forall k \Rightarrow \|\mathcal{S}-\mathcal{S}'\| \le \sqrt{N}\,B$	Sampling N times multiplies a uniform bound by √N.
`certified_cos_ode`	$\big(\sqrt{E_{S^c}}\|h_{S^c}\| + 2L_c\|h\|^2\big)^2 \le (1-c^2)\big(\|\mathcal{S}_0 h\| - L_c\|h\|^2\big)^2 \Rightarrow \cos\Delta(h) \ge c, \quad L_c = \sqrt{N} M_2 e^{2M_1 T}\dfrac{e^{M_1 T}-1}{M_1}$	Hypotheses: f is C^1 on U; on a convex K ⊆ U, ‖Df‖ ≤ M_1 and Df is M_2-Lipschitz; solutions exist for θ in a set containing the ball $\bar B(\theta_0, \rho_0)$, start from the same initial value and stay in K. Conclusion: the sensitivity $\mathcal{S}_0$ exists and, for every subset S and every ‖h‖ ≤ ρ_0 with ‖$\mathcal{S}_0$h‖ − L_c‖h‖² > 0, the displayed inequality certifies cos Δ(h) ≥ c.

Step 21. `ExactLinearCertificate` (13)

(A1) Certificates with the exact linear error ‖J(h − h_S)‖ instead of its energy bound, and (A2) a local version without Lipschitz constants.

Lean name	Statement	Meaning
`cos_certificate_of_bounds`	$0 < A \le \|a\|,\ \|a-b\| \le e,\ e^2 \le (1-c^2)A^2,\ c > 0 \Rightarrow b \neq 0,\ \cos(a,b) \ge c$	Generic certificate lemma.
`cosv_smul`	$\cos(ra, rb) = \cos(a, b) \quad (r > 0)$	The cosine is invariant under a common positive rescaling.
`continuousAt_cosv`	$(a, b) \mapsto \cos(a, b)$ is continuous at $a \neq 0,\ b \neq 0$	Continuity of the cosine.
`keepS_smul`	$(sh)_S = s\, h_S$	Restriction to S is linear.
`norm_linear_part_eq`	$\|Jh - Jh_S\| = \sqrt{\sum_i \big(\sum_{j \notin S} J_{ij} h_j\big)^2}$	The exact linear error is computable from J.
`finite_trajectory_error_exact`	$\|F(\theta_0 + h) - F(\theta_0 + h_S)\| \le \|J(h - h_S)\| + 2L_c\|h\|^2$	(A1) Finite-perturbation error with the exact linear term.
`exact_le_Rvar_bound`	$\|Jh - Jh_S\| \le \sqrt{E_{S^c}}\, \|h_{S^c}\|$	The exact bound is never worse than the energy bound.
`finite_cos_certificate_exact`	$\|Jh\| - L_c\|h\|^2 > 0,\ \big(\|J(h-h_S)\| + 2L_c\|h\|^2\big)^2 \le (1-c^2)\big(\|Jh\| - L_c\|h\|^2\big)^2 \Rightarrow \cos\Delta(h) \ge c$	(A1) Certificate of cos Δ with the exact linear error.
`certified_cos_ode_exact`	$\big(\|\mathcal{S}_0(h-h_S)\| + 2L_c\|h\|^2\big)^2 \le (1-c^2)\big(\|\mathcal{S}_0 h\| - L_c\|h\|^2\big)^2 \Rightarrow \cos\Delta(h) \ge c$	(A1) ODE version with the formally bounded L_c of certified_cos_ode.
`tendsto_cos_delta`	$Jh \neq 0,\ Jh_S \neq 0 \Rightarrow \lim_{s \to 0^+} \cos\Delta(sh) = \cos(Jh, Jh_S)$	(A2) For small perturbations, cos Δ tends to the linear cosine computed from J.
`eventually_cos_delta_gt`	$c < \cos(Jh, Jh_S) \Rightarrow \cos\Delta(sh) > c$ for all sufficiently small $s > 0$	(A2) Local certificate without L_c.
`exists_cos_delta_gt`	$c < \cos(Jh, Jh_S) \Rightarrow \exists s_0 > 0\ \forall s \in (0, s_0) :\ \cos\Delta(sh) > c$	Explicit form of the previous statement.
`local_cos_ode`	$\exists \mathcal{S}_0\ \forall S, h :\ c < \cos(\mathcal{S}_0 h, \mathcal{S}_0 h_S) \Rightarrow \exists s_0 > 0\ \forall s \in (0, s_0) :\ \cos\Delta_{\mathcal{F}}(sh) > c$	(A2) ODE version: if f is C^1 around the nominal trajectory and solutions exist near θ_0, the linear cosine computed from the sampled sensitivity certifies cos Δ for small perturbations.

Step 22. `NominalCertificate` **(1)**

Local certificate of cos Δ assuming only the nominal solution.

Lean name	Statement	Meaning
`local_cos_from_nominal`	$\exists x(\cdot;\theta),\ \exists \mathcal{S}_0 :\ \forall S, h :\ c < \cos(\mathcal{S}_0 h, \mathcal{S}_0 h_S) \Rightarrow \exists s_0 > 0\ \forall s \in (0, s_0) :\ \cos\Delta_{\mathcal{F}}(sh) > c$	Combines LocalExistence and ExactLinearCertificate: from the nominal solution alone, solutions near θ_0 and the sensitivity exist, and the linear cosine certifies cos Δ for small perturbations.

Part IV. Hypotheses of the benchmark models (kinetic models and reaction networks)

Step 23. `KineticRegularity` (11)

Hypothesis 1: rate laws (mass action, Michaelis–Menten, Hill, ...) define infinitely differentiable vector fields on their domain.

Notation: $\mathrm{MM}(V,K,x) = \dfrac{Vx}{K+x}$ (`mm V K x`): Michaelis–Menten law; $\mathrm{Hill}_m = \dfrac{Vx^m}{K^m+x^m}$ (`hill V K x m`, `hillR V K x h`): Hill law with integer (m) or real (h) exponent; $\mathrm{noDen}(e)$ (`e.noDen`): e has no division, real power or logarithm (mass action).

Lean name	Statement	Meaning
`KExpr.contDiffAt_eval`	$\mathrm{ok}_e(z) \Rightarrow e \in C^k$ near z $(\forall k)$	Every well-defined rate expression is smooth.
`KExpr.isOpen_ok`	$\{z : \mathrm{ok}_e(z)\}$ is open	The set where e is well defined is open.
`KExpr.ok_of_noDen`	$\mathrm{noDen}(e) \Rightarrow \mathrm{ok}_e(z)\ \forall z$	Mass-action expressions are defined everywhere.
`KExpr.ok_mm`	$K > 0,\ x \geq 0 \Rightarrow \mathrm{ok}(\mathrm{MM}(V,K,x))$	Michaelis–Menten is well defined for positive K and non-negative x.
`KExpr.ok_hill`	$K > 0,\ x \geq 0 \Rightarrow \mathrm{ok}(\mathrm{Hill}_m(V,K,x))$	Hill law with integer exponent.
`KExpr.ok_hillR`	$K > 0,\ x > 0 \Rightarrow \mathrm{ok}(\mathrm{Hill}_h(V,K,x))$	Hill law with real exponent requires x > 0.
`isOpen_domain`	$\mathcal{D}_F$ is open	The domain of a kinetic model is open.
`contDiffOn_field`	$f_F \in C^k(\mathcal{D}_F)$ $(\forall k)$	The vector field of a kinetic model is smooth on its domain.
`domain_eq_univ`	$\forall i:\ \mathrm{noDen}(F_i) \Rightarrow \mathcal{D}_F = \mathbb{R}^n \times \mathbb{R}^p$	Pure mass-action models are defined everywhere.
`kinetic_hasFDerivAt`	$(x_0(t),\theta_0) \in \mathcal{D}_F\ \forall t \Rightarrow \exists x(\cdot;\theta), S:\ S(0) = D_0,\ \dot{S} = \partial_x f_F\, S + \partial_\theta f_F,\ \partial_\theta x(t;\theta_0) = S(t)$	Existence near θ_0 and differentiability for every kinetic model whose nominal solution stays in the domain.
`kinetic_local_cos`	$\exists x(\cdot;\theta), \mathcal{S}_0:\ c < \cos(\mathcal{S}_0 h, \mathcal{S}_0 h_S) \Rightarrow \exists s_0 > 0\ \forall s \in (0,s_0):\ \cos \Delta_{\mathcal{F}}(sh) > c$	Local cos Δ certificate for kinetic models.

Step 24. `PositivityInvariance` (15)

Quasi-positive models keep concentrations non-negative.

Notation: $y^+ = \max(y,0)$ (`pos y`): componentwise positive part.

Lean name	Statement	Meaning
`pos_apply`	$(y^+)_i = \max(y_i, 0)$	Definition.
`pos_nonneg`	$y^+ \geq 0$	The positive part is non-negative.
`pos_eq_self`	$y \geq 0 \Rightarrow y^+ = y$	Non-negative vectors are unchanged.
`dist_pos_le`	$\|y^+ - y'^+\| \leq \|y - y'\|$	The positive part is 1-Lipschitz.
`sub_pos_apply`	$(y - y^+)_i = \min(y_i, 0)$	Negative part.
`pos_apply_eq_zero`	$y_i < 0 \Rightarrow (y^+)_i = 0$	Negative components are set to 0.

Lean name	Statement	Meaning
`isClosed_nonneg`	$\{y : \ y \geq 0\}$ is closed	The non-negative orthant is closed.
`hasDerivAt_negSq`	$\frac{d}{da}\min(a,0)^2 = 2\min(a,0)$	Derivative of the squared negative part.
`nonneg_core`	$x(a) \geq 0,\ \|v(x) - v(x^+)\| \leq K\|x - x^+\|,\ \big(x_i^+ = 0 \Rightarrow v_i(x^+) \geq 0\big) \Rightarrow x(t) \geq 0\ \forall t \in [a,b]$	Core argument: the squared negative part satisfies a Grönwall inequality with zero initial value.
`nonneg_of_quasiPositive`	$\big(y \geq 0,\ y_i = 0 \Rightarrow v_i(y) \geq 0\big),\ x(0) \geq 0 \Rightarrow x(t) \geq 0\ \forall t \in [0,T]$	Quasi-positive C^1 vector fields keep the orthant invariant.
`kinetic_nonneg`	$\mathbb{R}^n_{\geq 0} \times \{\theta\} \subseteq \mathcal{D}_F,\ f_F$ quasi-positive, $x(0) \geq 0 \Rightarrow x(t) \geq 0,\ (x(t),\theta) \in \mathcal{D}_F$	Kinetic version.
`kinetic_hasFDerivAt_of_nonneg`	$\Rightarrow \exists x(\cdot;\theta), S :\ S(0) = D_0,\ \partial_\theta x(t;\theta_0) = S(t)$	Kinetic models defined on the orthant: positivity plus existence near θ_0 and differentiability.
`exampleModel_orth`	$\dot{x} = \theta_0 - \frac{\theta_1 x}{\theta_2 + x} :\ \theta_2 > 0,\ x \geq 0 \Rightarrow (x,\theta) \in \mathcal{D}$	Worked example (Michaelis–Menten degradation): well defined on the orthant.
`exampleModel_qp`	$\theta_0 \geq 0,\ x = 0 \Rightarrow \dot{x} \geq 0$	The example is quasi-positive.
`exampleModel_nonneg`	$\theta_0 \geq 0,\ \theta_2 > 0,\ x(0) \geq 0 \Rightarrow x(t) \geq 0$	The example stays non-negative.

Step 25. `GlobalExistence` (11)

Condition C4: global existence of the nominal solution under a linear growth bound on the non-negative orthant.

Notation: $\mathrm{clamp}_B(z)_i = \max(0, \min(B, z_i))$ (`clampB B z`): truncation to the box $[0, \mathrm{B}]^n$; $[0,B]^n$ (`box B`): box.

Lean name	Statement	Meaning
`clampB_apply`	$\mathrm{clamp}_B(z)_i = \max(0, \min(B, z_i))$	Definition.
`clampB_nonneg`	$\mathrm{clamp}_B(z) \geq 0$	Truncation is non-negative.
`clampB_le`	$B \geq 0 \Rightarrow \mathrm{clamp}_B(z)_i \leq B$	Truncation is bounded by B.
`clampB_pos`	$\mathrm{clamp}_B(z^+) = \mathrm{clamp}_B(z)$	Truncation ignores the negative part.
`clampB_eq`	$0 \leq z \leq B \Rightarrow \mathrm{clamp}_B(z) = z$	Truncation fixes points of the box.
`dist_clampB_le`	$\|\mathrm{clamp}_B(y) - \mathrm{clamp}_B(y')\| \leq \|y - y'\|$	Truncation is 1-Lipschitz.
`clampB_mem`	$\mathrm{clamp}_B(z) \in [0,B]^n$	Truncation lands in the box.
`convex_box`	$[0,B]^n$ is convex	Convexity of the box.
`isCompact_box`	$[0,B]^n$ is compact	Compactness of the box.
`lipschitzOn_box`	$v \in C^1(U),\ [0,B]^n \subseteq U \Rightarrow \exists L :$ v is L-Lipschitz on $[0,B]^n$	C^1 vector fields are Lipschitz on compact boxes.
`exists_global_solution`	v quasi-positive, $\sum_i w_i v_i(z) \leq a + b\sum_i w_i z_i\ (z \geq 0),\ x^0 \geq 0 \Rightarrow \exists x :$ $x(0) = x^0,\ \dot{x} = v(x),\ x(t) \geq 0\ \forall t \in [0,T]$	Global existence for every horizon T: the weighted sum Σ w_i x_i grows at most exponentially, so the solution cannot blow up.

Step 26. `StrictExistence` (12)

Global existence when some species must stay strictly positive (for example denominators x_j).

Notation: $\ell_i = \varepsilon$ $(i \in \Sigma)$, 0 $(i \notin \Sigma)$ (`lo sx ε i`): lower corner of the box; $\mathrm{clamp}_Q(z)_i = \max(\ell_i, \min(B, z_i))$ (`clampQ sx ε B z`): truncation to the box Q = Π_i [ℓ_i, B]; Q (`boxQ sx ε B`): box.

Lean name	Statement	Meaning
`lo_nonneg`	$\varepsilon \geq 0 \Rightarrow \ell_i \geq 0$	Lower corner is non-negative.
`lo_le`	$0 \leq \varepsilon \leq B \Rightarrow \ell_i \leq B$	Lower corner is below B.
`clampQ_apply`	$\mathrm{clamp}_Q(z)_i = \max(\ell_i, \min(B, z_i))$	Definition.
`clampQ_mem`	$\mathrm{clamp}_Q(z) \in Q$	Truncation lands in the box.
`boxQ_sPos`	$\varepsilon > 0,\ z \in Q \Rightarrow z \in P_\Sigma$	The box lies in the strict region.
`clampQ_pos`	$\mathrm{clamp}_Q(z^+) = \mathrm{clamp}_Q(z)$	Truncation ignores the negative part.
`clampQ_eq`	$\ell \leq z \leq B \Rightarrow \mathrm{clamp}_Q(z) = z$	Truncation fixes points of the box.
`dist_clampQ_le`	$\lVert \mathrm{clamp}_Q(y) - \mathrm{clamp}_Q(y') \rVert \leq \lVert y - y' \rVert$	Truncation is 1-Lipschitz.
`convex_boxQ`	Q is convex	Convexity.
`isCompact_boxQ`	Q is compact	Compactness.
`lipschitzOn_boxQ`	$v \in C^1(U),\ Q \subseteq U \Rightarrow v$ Lipschitz on Q	Lipschitz on the box.
`exists_global_solution_strict`	v quasi-positive on P_Σ, $v_i(z) \geq -C_B z_i$ $(i \in \Sigma,\ z \leq B)$, $\sum_i w_i v_i \leq a + b \sum_i w_i z_i$, $x^0 \in P_\Sigma \Rightarrow \exists x :\ x(t) \in P_\Sigma\ \forall t \in [0, T]$	Global existence staying in the strict region: species in Σ decay at most exponentially and never reach 0.

Step 27. `KineticCheck` (19)

Verified Boolean checkers that Lean runs on each translated model (SBML → KExpr → Lean).

Notation: isNonneg, isPos, okOrth (`isNonneg pos e, isPos pos e, okOrth pos e`): syntactic tests: e ≥ 0, e > 0, e well defined on the orthant; $\mathrm{vanishes}_i$, qp_i (`vanishes i e, qp pos i e`): e = 0 when x_i = 0; quasi-positivity of component i; checkModel, checkInit (`checkModel pos F, checkInit pos G`): all components pass okOrth and qp; initial expressions G pass okOrth and isNonneg.

Lean name	Statement	Meaning
`KExpr'.isNonneg_sound`	$y \geq 0,\ \theta_P > 0,\ \mathrm{isNonneg}(e) \Rightarrow e(y, \theta) \geq 0$	Soundness of the non-negativity checker.
`KExpr'.isPos_sound`	$y \geq 0,\ \theta_P > 0,\ \mathrm{isPos}(e) \Rightarrow e(y, \theta) > 0$	Soundness of the positivity checker.
`KExpr'.okOrth_sound`	$y \geq 0,\ \theta_P > 0,\ \mathrm{okOrth}(e) \Rightarrow \mathrm{ok}_e(y, \theta)$	Soundness of the domain checker.
`KExpr'.vanishes_sound`	$y_i = 0,\ \mathrm{vanishes}_i(e) \Rightarrow e(y, \theta) = 0$	Soundness of the vanishing checker.
`KExpr'.qp_sound`	$y \geq 0,\ y_i = 0,\ \theta_P > 0,\ \mathrm{qp}_i(e) \Rightarrow e(y, \theta) \geq 0$	Soundness of the quasi-positivity checker.
`checkModel_spec`	$\mathrm{checkModel}(F) \Rightarrow \mathrm{okOrth}(F_i) \wedge \mathrm{qp}_i(F_i)\ \forall i$	Unfolding of the model checker.
`checkModel_sound`	$\mathrm{checkModel}(F),\ \theta_P > 0 \Rightarrow \mathbb{R}^n_{\geq 0} \times \{\theta\} \subseteq \mathcal{D}_F\ \wedge\ \big(y \geq 0, y_i = 0 \Rightarrow f_{F,i}(y, \theta) \geq 0\big)$	A checked model is well defined and quasi-positive on the orthant.
`checked_model_hasFDerivAt`	$x_0(0) \geq 0 \Rightarrow x_0(t) \geq 0,\ (x_0(t), \theta_0) \in \mathcal{D}_F,\ \exists x(\cdot;\theta), S :\ S(0) = D_0,\ \partial_\theta x(t; \theta_0) = S(t)$	Checked model: positivity, existence near θ_0 and differentiability.
`checked_segments_hasFDerivAt`	$y_{k+1}(0) = y_k(L_k) \Rightarrow \exists Y_k(\cdot;\theta)$: segments solve and are differentiable in θ	Same for models simulated in segments (condition changes at fixed times).
`qvec_apply`	$\iota(q)_i = q_i$	Rational vectors as real vectors.

Lean name	Statement	Meaning
`posParams_qvec`	$\text{checkPosParams}(\theta_q) \Rightarrow \iota(\theta_q)_P > 0$	The rational parameter values are positive where declared.
`nonneg_qvec`	$\text{checkNonneg}(x_q) \Rightarrow \iota(x_q) \geq 0$	The rational initial values are non-negative.
`checked_model_final`	$\text{checkModel} \wedge \text{checkPosParams} \wedge \text{checkNonneg} \Rightarrow x_0 \geq 0,\ \text{in } \mathcal{D}_F,\ \exists x(\cdot;\theta), S :\ S(0) = 0,\ \partial_\theta x(t;\theta_q) = S(t)$	End-to-end theorem for a checked model with rational parameters and initial values.
`checked_segments_final`	same, for segmented models	End-to-end theorem for segmented models.
`checkInit_spec`	$\text{checkInit}(G) \Rightarrow \text{okOrth}(G_i) \wedge \text{isNonneg}(G_i)$	Unfolding of the initial-value checker.
`nonneg_zero`	$0 \geq 0$	The zero vector is non-negative.
`init_differentiable`	$\text{checkInit}(G) \Rightarrow \theta \mapsto G(\theta)$ differentiable at θ_0	Parameter-dependent initial values are differentiable.
`init_nonneg`	$\text{checkInit}(G) \Rightarrow G(\theta_0) \geq 0$	Parameter-dependent initial values are non-negative.
`checked_model_final_init`	$x(0;\theta) = G(\theta) :\ \ldots,\ S(0) = \partial_\theta G(\theta_q)$	End-to-end theorem with parameter-dependent initial values.

Step 28. `KineticNetwork` (18)

Reaction networks f_i = Σ_k ν_ik r_k: complete automatic check including global existence (C4).

Notation: checkNet, checkGrowth (`checkNet pos Rx, checkGrowth pos c Rx`): every reaction passes the domain and quasi-positivity test; every reaction passes the linear-growth test with weights w; $\ell_e(\theta)$ (`lowval pos θ e`): positive constant lower bound of e (from lowerpos); A_e, B_e (`linBound pos θ e`): coefficients of a linear bound e ≤ A + B Σ_j y_j.

Lean name	Statement	Meaning
`eval_netExpr`	$f_i(z) = \sum_k \nu_{ik}\, r_k(z)$	The network vector field is the stoichiometric sum.
`ok_netExpr`	$\forall k :\ \text{ok}_{r_k}(z) \Rightarrow \text{ok}_{f_i}(z)$	Well-defined rates give a well-defined field.
`coef_mul_nonneg`	$\text{termOK}(r_k, \nu_k),\ y \geq 0,\ y_i = 0 \Rightarrow \nu_{ik}\, r_k(y,\theta) \geq 0$	Each reaction respects quasi-positivity (consumption terms vanish when the species is absent).
`checkNet_sound`	$\text{checkNet} \Rightarrow \mathbb{R}^n_{\geq 0} \times \{\theta\} \subseteq \mathcal{D} \ \wedge\ f$ quasi-positive	Soundness of the network checker.
`eval_varfree`	e without state variables $\Rightarrow e(y,\theta) = e(y',\theta)$	Constant expressions do not depend on the state.
`ssum_nonneg`	$y \geq 0 \Rightarrow \sum_j y_j \geq 0$	Sum of a non-negative vector.
`lowerpos_sound`	$\text{lowerpos}(e),\ y \geq 0 \Rightarrow 0 < \ell_e(\theta) \leq e(y,\theta)$	Denominators bounded away from 0.
`linBound_sound`	$\text{linOK}(e),\ y \geq 0 \Rightarrow A_e, B_e \geq 0,\ e(y,\theta) \leq A_e + B_e \sum_j y_j$	Linear upper bound of a rate law.
`sum_c_coef`	$\sum_i w_i \nu_{ik} = \langle w, \nu_k \rangle$	Weighted stoichiometry.
`sum_c_netExpr`	$\sum_i w_i f_i = \sum_k \langle w, \nu_k \rangle\, r_k$	Weighted sum of the vector field.

Lean name	Statement	Meaning
`checkGrowth_sound`	checkGrowth $\Rightarrow \exists a, b \geq 0$: $\sum_i w_i f_i(y,\theta) \leq a + b \sum_i w_i y_i \ (y \geq 0)$	Soundness of the growth checker.
`checkNet_append`	checkNet$(A +\!\!+ B) =$ checkNet$(A) \wedge$ checkNet(B)	The check can be split over parts of a large model.
`checkGrowth_append`	checkGrowth$(A +\!\!+ B) =$ checkGrowth$(A) \wedge$ checkGrowth(B)	Same for the growth check.
`c_ge_one`	checkGrowth $\Rightarrow w_i \geq 1$	Weights are at least 1.
`network_exists`	checkNet, checkGrowth, $x^0 \geq 0 \Rightarrow \exists x$: $\dot{x} = f(x,\theta_0),\ x(t) \geq 0\ \forall t \in [0,T]$	Global existence for checked networks.
`network_final`	$\Rightarrow \exists x_0 \geq 0$ on $[0,T],\ \exists x(\cdot;\theta), S:\ S(0) = 0,\ \partial_\theta x(t;\theta_q) = S(t)$	End-to-end theorem for a network with rational parameters and initial values. Used by the models Bertozzi, Blasi, Boehm, Brannmark, Bruno, Chen, Froehlich, Isensee, Lang, Laske, Raimundez, Sneyd, Weber, Zhao, Zheng.
`network_final_init`	$x(0;\theta) = G(\theta) :\ \ldots,\ S(0) = \partial_\theta G(\theta_q)$	Same with parameter-dependent initial values. Used by the models Bachmann, Fujita, Lucarelli, Raia, SalazarCavazos.
`network_segments_final`	same, for segmented models	Same for models simulated in segments. Used by the models Giordano.

Step 29. `RiccatiNetwork` (9)

Existence on a finite horizon under quadratic growth (Riccati bound); used for Crauste.

Notation: $\varphi(y) = \sum_i w_i y_i$ (`phi c y`): weighted total; Q (`Qof (tot c θq Rx)`): Riccati constant max(A, B/2, C) of the bound Σ w_i f_i $\leq$ A + Bφ + Cφ^2.

Lean name	Statement	Meaning
`exists_solution_riccati`	$\sum_i w_i v_i(z) \leq Q(\varphi(z)+1)^2,\ (1+\delta)Q(\varphi(x^0)+1)T < 1 \Rightarrow \exists x \geq 0$ on $[0,T]$	Quasi-positive field with quadratic growth: the solution exists up to the Riccati blow-up time.
`qval_sound`	qc$(e) \Rightarrow e(y, \iota(\theta_q)) =$ qval$(e) \in \mathbb{Q}$	Constant expressions evaluate exactly in rational arithmetic.
`qnn_sound`	qnn$(e) \Rightarrow e =$ qval$(e) \geq 0$	Non-negative rational constants.
`var_le_phi`	$w > 0,\ y \geq 0 \Rightarrow y_j \leq \varphi(y)/w_j$	Each species is bounded by the weighted total.
`linV_sound`	linC$(e) \Rightarrow 0 \leq e \leq A + B\varphi(y),\ A, B \geq 0$	Linear bound of a rate law.
`prodV_sound`	$0 \leq a \leq A_1 + B_1\varphi,\ 0 \leq b \leq A_2 + B_2\varphi \Rightarrow 0 \leq ab \leq A + B\varphi + C\varphi^2$	Product of two linear bounds is a quadratic bound.
`quadV_sound`	quadC$(e) \Rightarrow 0 \leq e \leq A + B\varphi + C\varphi^2$	Quadratic bound of a rate law.
`tot_sound`	$\sum_k \langle w, \nu_k \rangle r_k \leq A + B\varphi + C\varphi^2$	Quadratic bound of the weighted vector field (sum over reactions with positive weight).

Lean name	Statement	Meaning
`riccati_final`	checkRiccati : $1.1\,Q(\varphi(x_q)+1)T_q < 1,\ T \le T_q \Rightarrow \exists x_0 \ge 0$ on $[0,T],\ \exists x(\cdot;\theta), S :\ \partial_\theta x(t;\theta_q) = S(t)$	End-to-end theorem on a finite horizon (Crauste: T $\le$ 1). Used by the models Crauste.

Step 30. `StrictNetwork` (46)

Reaction networks with strictly positive domains (e.g. βSI/N, Hill laws with real exponents): checkers and their soundness.

Notation: UB(θ) (`UB ub` θ): declared lower/upper bounds of parameters hold; PP(θ) (`PP pos ub` θ): θ_P > 0 and UB(θ); sNonneg, snp, expLe1, sPos, sOk: syntactic tests on P_Σ: e ≥ 0, e ≤ 0, $0 \le$ e ≤ 1 (exponential of a non-positive term), e > 0, e well defined; Bnd(e) (`BndB sx` θ `bx e`): $\forall$B $\exists$M: $0 \le$ e(y,θ) $\le$ M on P_Σ (and on [0,B]^n if bx); Cons$_i$(e) (`ConsB sx` θ `i e`): $\forall$B $\exists$K: $0 \le$ e(y,θ) $\le$ K$\cdot$y_i on P_$\Sigma \cap$ [0,B]^n (consumption term proportional to y_i); checkNetS, checkGrowthS (`checkNetS pos ub sx Rx, checkGrowthS pos ub c Rx`): network checks on the strict region.

Lean name	Statement	Meaning
`keq_sound`	keq$(a,b) \Rightarrow a = b$	Syntactic equality is sound.
`qge1_sound`	qge1$(e) \Rightarrow e \ge 1$	Rational constant ≥ 1.
`qgt1_sound`	qgt1$(e) \Rightarrow e > 1$	Rational constant > 1.
`addlb_sound`	$\theta_j \ge r \ge -q \Rightarrow q + \theta_j \ge 0$	Sum of a constant and a parameter with a declared lower bound.
`subub_sound`	$0 < \theta_j \le r \le q \Rightarrow 0 \le q - \theta_j \le q$	Difference of a constant and a parameter with a declared upper bound.
`ub_qvec`	checkUB$(\theta_q) \Rightarrow$ UB$(\iota(\theta_q))$	The rational parameters satisfy the declared bounds.
`sign_sound`	sNonneg$(e) \Rightarrow e \ge 0,$ snp$(e) \Rightarrow e \le 0,$ expLe1$(e) \Rightarrow 0 \le e \le 1$	Joint soundness of the sign checkers (y ≥ 0, PP(θ)).
`sNonneg_sound`	sNonneg$(e) \Rightarrow e(y,\theta) \ge 0$	Non-negativity checker.
`expLe1_sound`	expLe1$(e) \Rightarrow 0 \le e(y,\theta) \le 1$	Exponential of a non-positive term.
`sPos_sound`	$y \in P_\Sigma,$ sPos$(e) \Rightarrow e(y,\theta) > 0$	Positivity checker on the strict region.
`sOk_sound`	$y \in P_\Sigma,$ sOk$(e) \Rightarrow$ ok$_e(y,\theta)$	Domain checker on the strict region.
`snp_sound`	snp$(e) \Rightarrow e(y,\theta) \le 0$	Non-positivity checker.
`summand_sound`	summand$(X,d) \Rightarrow 0 \le X \le d$	X is a non-negative summand of d, so X/d $\in$ [0, 1].
`bndB_const`	e constant, $e \ge 0 \Rightarrow$ Bnd(e)	Constants are bounded.
`bndB_div_lowerpos`	Bnd$(a),\ d \ge \ell > 0 \Rightarrow$ Bnd(a/d)	Division by a term bounded away from 0.
`bndB_summand`	summand$(a,d) \Rightarrow$ Bnd(a/d)	Saturating terms a/(a + ...) are bounded.
`bndB_mul`	Bnd(a), Bnd$(b) \Rightarrow$ Bnd(ab)	Products of bounded terms.
`bndB_add`	Bnd(a), Bnd$(b) \Rightarrow$ Bnd$(a+b)$	Sums of bounded terms.
`bndB_frac_mul_l`	Bnd(u/d), Bnd$(v) \Rightarrow$ Bnd(uv/d)	Bounded fraction times bounded factor.
`bndB_frac_mul_r`	Bnd(u), Bnd$(v/d) \Rightarrow$ Bnd(uv/d)	Same, factor on the left.
`bndB_pair`	Bnd(a_1/d_1), Bnd$(a_2/d_2) \Rightarrow$ Bnd$\big(a_1a_2/(d_1d_2)\big)$	Product of two bounded fractions.

Lean name	Statement	Meaning
`bndB_pair'`	$\mathrm{Bnd}(a_1/d_2), \mathrm{Bnd}(a_2/d_1) \Rightarrow \mathrm{Bnd}\big(a_1a_2/(d_1d_2)\big)$	Same, crossed pairing.
`consB_mul_l`	$\mathrm{Cons}_i(a), \mathrm{Bnd}(b) \Rightarrow \mathrm{Cons}_i(ab)$	Proportional to y_i times bounded.
`consB_mul_r`	$\mathrm{Bnd}(a), \mathrm{Cons}_i(b) \Rightarrow \mathrm{Cons}_i(ab)$	Same, factor on the left.
`consB_div_lowerpos`	$\mathrm{Cons}_i(a),\ d \geq \ell > 0 \Rightarrow \mathrm{Cons}_i(a/d)$	Proportional term divided by a positive term.
`bndB_congr`	$e \equiv e',\ \mathrm{Bnd}(e') \Rightarrow \mathrm{Bnd}(e)$	Bounds depend only on the value of the expression.
`consB_congr`	$e \equiv e',\ \mathrm{Cons}_i(e') \Rightarrow \mathrm{Cons}_i(e)$	Same for proportional bounds.
`eval_pair`	$\dfrac{a_1a_2}{d_1d_2} = \dfrac{a_1}{d_1} \cdot \dfrac{a_2}{d_2}$	Algebraic identity used to pair factors.
`eval_pair'`	$\dfrac{a_1a_2}{d_1d_2} = \dfrac{a_1}{d_2} \cdot \dfrac{a_2}{d_1}$	Crossed pairing.
`consB_var`	$\mathrm{Cons}_i(y_i)$	A species is proportional to itself.
`consB_npow`	$k \geq 1,\ \mathrm{Cons}_i(a),\ \mathrm{Bnd}(a) \Rightarrow \mathrm{Cons}_i(a^k)$	Powers of proportional terms.
`bndB_npow`	$\mathrm{Bnd}(a) \Rightarrow \mathrm{Bnd}(a^k)$	Powers of bounded terms.
`bndB_exp_np`	$a \leq 0 \Rightarrow \mathrm{Bnd}(e^a)$	Exponential of a non-positive term is in (0, 1].
`bndB_var`	$\mathrm{Bnd}(y_j)$ on $[0, B]^n$	Species are bounded on the box.
`bq_of`	$\big(\mathrm{Bnd}(a) \wedge d \geq \ell > 0\big) \vee \mathrm{satb} \vee \mathrm{bpair} \Rightarrow \mathrm{Bnd}(a/d)$	Assembles the three ways of bounding a fraction.
`cq_of`	$\big(\mathrm{Cons}_i(a) \wedge d \geq \ell > 0\big) \vee \mathrm{cpair} \Rightarrow \mathrm{Cons}_i(a/d)$	Assembles the ways of proving a proportional fraction.
`bnd_sound`	$\mathrm{bnd}, \mathrm{satb}, \mathrm{bpair} \Rightarrow \mathrm{Bnd};\quad \mathrm{cons}, \mathrm{cpair} \Rightarrow \mathrm{Cons}_i$	Joint soundness of the bound checkers.
`term_qp`	$\mathrm{termOKS}(r_k, \nu_k),\ y \in P_\Sigma,\ y_i = 0 \Rightarrow \nu_{ik} r_k(y, \theta) \geq 0$	Quasi-positivity of each reaction on the strict region.
`term_decay`	$i \in \Sigma \Rightarrow \forall B\ \exists K \geq 0 :\ \nu_{ik} r_k(y, \theta) \geq -K y_i\ (y \in P_\Sigma,\ y \leq B)$	Consumption of a strictly positive species is at most proportional to it.
`checkNetS_sound`	$\mathrm{checkNetS} \Rightarrow P_\Sigma \times \{\theta\} \subseteq \mathcal{D},\ f$ quasi-positive, $f_i \geq -C_B y_i\ (i \in \Sigma)$	Soundness of the strict network checker.
`term_growth`	$\mathrm{growthOKS} \Rightarrow \exists A, B \geq 0 :$ $\langle w, \nu_k \rangle r_k(y, \theta) \leq A + B \sum_j y_j$	Growth bound for one reaction.
`checkGrowthS_sound`	$\mathrm{checkGrowthS} \Rightarrow \exists a, b \geq 0 :\ \sum_i w_i f_i \leq a + b \sum_i w_i y_i\ (y \in P_\Sigma)$	Growth bound for the network.
`c_ge_oneS`	$\mathrm{checkGrowthS} \Rightarrow w_i \geq 1$	Weights are at least 1.
`strict_exists`	$x^0 \in P_\Sigma \Rightarrow \exists x :\ \dot{x} = f(x, \theta_0),\ x(t) \in P_\Sigma\ \forall t \in [0, T]$	Global existence in the strict region.
`init_sPos`	$\mathrm{checkInitS}(G),\ \mathrm{PP}(\theta_0) \Rightarrow G(\theta_0) \in P_\Sigma$	Initial values lie in the strict region.
`strict_final_init`	$\Rightarrow x_0(t) \in P_\Sigma \cap \mathcal{D},\ \exists x(\cdot;\theta), S :\ S(0) = \partial_\theta G(\theta_q),\ \partial_\theta x(t;\theta_q) = S(t)$	End-to-end theorem for networks with strictly positive domains. Used by the models Armistead, Borghans, Elowitz, Okuonghae, Rahman.

Step 31. `IntervalInit` **(5)**

Parameter-dependent initial values certified with rational interval arithmetic.

Notation: $[l, u]$ (`ival θq e = some (l, u)`): rational interval enclosing the value of e at θ_q; checkInitI (`checkInitI sx θq G`): interval check that G(θ_q) ≥ 0 and > 0 on Σ.

Lean name	Statement	Meaning
`mul_mem_corners`	$x \in [a_1, a_2],\ y \in [b_1, b_2] \Rightarrow \min_{i,j} a_i b_j \leq xy \leq \max_{i,j} a_i b_j$	Interval multiplication rule.
`imul_sound`	$x \in a,\ y \in b \Rightarrow xy \in a \otimes b$	Rational interval product encloses the real product.
`ival_sound`	$\mathrm{ival}(e) = [l, u] \Rightarrow \mathrm{ok}_e(y, \iota(\theta_q)) \ \wedge\ l \leq e(y, \iota(\theta_q)) \leq u$	Interval evaluation is sound (real powers restricted to square roots).
`initI_sound`	$\mathrm{checkInitI}(G) \Rightarrow (0, \iota(\theta_q)) \in \mathcal{D}_G,\ G(\iota(\theta_q)) \in P_\Sigma$	The initial values are well defined and lie in the strict region.
`strict_final_initI`	$x(0;\theta) = G(\theta) :\ x_0(t) \in P_\Sigma \cap \mathcal{D},\ \exists x(\cdot;\theta), S :\ S(0) = \partial_\theta G(\theta_q),\ \partial_\theta x(t;\theta_q) = S(t)$	End-to-end theorem with initial values checked by interval arithmetic. Used by the models Fiedler.

Supplementary Note 4. The verified benchmark models

This note lists, for each of the 28 benchmark models verified in Lean, the data contained in its file in `lean_proofs/Models/`, the checks executed by the Lean kernel on those data and the final theorem that results. Each file was generated from the SBML model and the PEtab parameter table of the benchmark by `certificados/sbml_to_lean.py`, with every number written as an exact rational. The checks are Boolean functions proved sound in steps 27–31 of Supplementary Note 2; each is evaluated by the Lean kernel (`decide +kernel`), so its value `true` is part of the proof. The final theorem `Models.<model>.final` then has no pending hypotheses: it states that the model satisfies the assumptions of Parts II–III (smooth vector field on a domain containing the trajectory, existence of the nominal solution on the horizon, non-negativity, and differentiability of the solution with respect to the parameters). `#print axioms` for every final theorem reports only `propext`, `Classical.choice` and `Quot.sound` (`CheckModelAxioms.lean`). Table S6 summarises the models, the next subsections describe the independent check of the translation, a complete example (Boehm) and the benchmark problems that were not verified, and the last subsections give the data of each model.

Supplementary Table S6. Benchmark models verified in Lean. n: states (species, plus time when a rate law depends on it); p: parameters; R: rate terms (reactions after splitting reversible ones); $|\Sigma|$: species proved strictly positive; time: compilation time of the model files (s); fidelity: largest relative difference between the Lean translation and the SBML model evaluated by libRoadRunner (vector field, parameter values and initial values).

Model	n	p	R	$\lvert\Sigma\rvert$	Horizon	Final theorem	Time	Fidelity
Armistead_Cell DeathDis2024	4	10	8	4	all $T \geq 0$	`strict_final_init`	5	0
Bachmann_MSB2011	25	27	43	–	all $T \geq 0$	`network_final_init`	12	3×10^{-15}
Bertozzi_PNAS2020	3	0	2	–	all $T \geq 0$	`network_final`	188	0
Blasi_Cell Systems2016	16	8	36	–	all $T \geq 0$	`network_final`	8	4×10^{-15}
Boehm_JProteome Res2014	9	6	10	–	all $T \geq 0$	`network_final`	7	4×10^{-16}

Supplementary Table S6. Benchmark models verified in Lean. n: states (species, plus time when a rate law depends on it); p: parameters; R: rate terms (reactions after splitting reversible ones); $|\Sigma|$: species proved strictly positive; time: compilation time of the model files (s); fidelity: largest relative difference between the Lean translation and the SBML model evaluated by libRoadRunner (vector field, parameter values and initial values).

Model	n	p	R	$\lvert\Sigma\rvert$	Horizon	Final theorem	Time	Fidelity
Borghans_Biophys Chem1997	3	20	9	3	all $T \geq 0$	`strict_final_init`	5	2×10^{-16}
Brannmark_ JBC2010	9	15	14	–	all $T \geq 0$	`network_final`	6	1×10^{-13}
Bruno_JExpBot2016	7	6	6	–	all $T \geq 0$	`network_final`	4	2×10^{-16}
Chen_MSB2009	501	152	1,243	–	all $T \geq 0$	`network_final`	1080	8×10^{-14}
Crauste_Cell Systems2017	5	12	12	–	$T \leq 1$	`riccati_final`	6	0
Elowitz_Nature2000	8	18	18	8	all $T \geq 0$	`strict_final_init`	6	2×10^{-16}
Fiedler_BMCSyst Biol2016	7	12	8	6	all $T \geq 0$	`strict_final_initI`	15	0
Froehlich_Cell Systems2018	1,228	4,088	4,487	–	all $T \geq 0$	`network_final`	2021	pending
Fujita_SciSignal2010	9	16	13	–	all $T \geq 0$	`network_final_init`	5	1×10^{-16}
Giordano_ Nature2020	10	43	96	–	6 segments	`network_segments_final`	8	4×10^{-16}
Isensee_JCB2018	25	32	34	–	all $T \geq 0$	`network_final`	8	1×10^{-14}
Lang_PLOSComput Biol2024	124	164	663	–	all $T \geq 0$	`network_final`	264	2×10^{-15}
Laske_PLOSComput Biol2019	34	6	51	–	all $T \geq 0$	`network_final`	10	0
Lucarelli_Cell Systems2018	33	69	67	–	all $T \geq 0$	`network_final_init`	26	4×10^{-15}
Okuonghae_Chaos SolitonsFractals2020	8	14	11	6	all $T \geq 0$	`strict_final_init`	5	7×10^{-16}
Rahman_MBS2016	7	9	19	4	all $T \geq 0$	`strict_final_init`	5	1×10^{-15}
Raia_Cancer Research2011	14	18	14	–	all $T \geq 0$	`network_final_init`	5	6×10^{-16}
Raimundez_ PCB2020	22	57	91	–	all $T \geq 0$	`network_final`	15	1×10^{-12}
SalazarCavazos_ MBoC2020	75	6	457	–	all $T \geq 0$	`network_final_init`	318	2×10^{-13}
Sneyd_PNAS2002	6	14	16	–	all $T \geq 0$	`network_final`	5	4×10^{-15}
Weber_BMC2015	7	26	18	–	all $T \geq 0$	`network_final`	6	0
Zhao_QuantBiol2020	4	21	3	–	all $T \geq 0$	`network_final`	5	0
Zheng_PNAS2012	15	45	44	–	all $T \geq 0$	`network_final`	7	1×10^{-15}

Agreement of the translation with the SBML models

Lean checks the expressions it is given; whether these expressions are the model of the benchmark is a separate question. The script `certificados/verifica_traduccion.py` answers it without using the translator: it reads the rate laws, the stoichiometry, the parameter values θ_q and the initial values

directly from the Lean files, evaluates the vector field in floating point, and compares it with the derivatives computed by libRoadRunner from the original SBML model with the nominal parameters of the PEtab table, at the initial state and at five random positive states. It also compares θ_q with the PEtab nominal values and the initial values (or $G(\theta_q)$) with those of the SBML model. Over the 27 models checked so far (the check of Froehlich is still running), the largest relative difference is 1×10^{-12}, i.e. agreement to rounding error (Table S6, last column). As a control, changing one parameter value, one initial value or one stoichiometric coefficient of the Lean file by 1% is detected (relative differences of 10^{-2}). The comparison covers the model without condition-specific overrides, which is what the Lean files describe; for models defined by segments it covers the first segment.

A complete example: Boehm_JProteomeRes2014

The model of STAT5 dimerisation by Boehm et al. is small enough to follow every step. Its file is `lean_proofs/Models/Boehm_JProteomeRes2014.lean` (about 60 lines).

1. Model. The SBML model has 8 species, and the stimulus decays as $e^{-\theta_1 \tau}$, where τ is time; the translator therefore adds τ as a ninth state with $\dot{\tau} = 1$. The six estimated parameters are $\theta_1 =$ Epo_degradation_BaF3 (0.02698), $\theta_2 =$ k_exp_hetero (1.001×10^{-5}), $\theta_3 =$ k_exp_homo (0.00617), $\theta_4 =$ k_imp_hetero (0.01637), $\theta_5 =$ k_imp_homo (9.775×10^4), $\theta_6 =$ k_phos (1.577×10^4). The vector field is $f_i = \sum_k \nu_{ik} v_k$ with the following rate terms v_k and stoichiometric coefficients ν_{ik} (the constants come from compartment volumes and fixed parameters of the SBML model):

k	Rate term v_k	Stoichiometry $\nu_{\cdot k}$
1	$\frac{7}{5} \frac{1}{8000000}\, e^{(-\theta_1)\,\tau}\, \mathrm{STAT5A}^2\, \theta_6$	STAT5A: -10/7, pApA: 5/7
2	$\frac{7}{5} \frac{1}{8000000}\, e^{(-\theta_1)\,\tau}\, \mathrm{STAT5A}\, \mathrm{STAT5B}\, \theta_6$	STAT5A: -5/7, STAT5B: -5/7, pApB: 5/7
3	$\frac{9}{20}\, \theta_3\, \mathrm{nucpApA}$	STAT5A: 10/7, nucpApA: -20/9
4	$\frac{9}{20}\, \theta_2\, \mathrm{nucpApB}$	STAT5A: 5/7, STAT5B: 5/7, nucpApB: -20/9
5	$\frac{7}{5} \frac{1}{8000000}\, e^{(-\theta_1)\,\tau}\, \mathrm{STAT5B}^2\, \theta_6$	STAT5B: -10/7, pBpB: 5/7
6	$\frac{9}{20}\, \theta_3\, \mathrm{nucpBpB}$	STAT5B: 10/7, nucpBpB: -20/9
7	$\frac{7}{5}\, \theta_4\, \mathrm{pApB}$	pApB: -5/7, nucpApB: 20/9
8	$\frac{7}{5}\, \theta_5\, \mathrm{pApA}$	pApA: -5/7, nucpApA: 20/9
9	$\frac{7}{5}\, \theta_5\, \mathrm{pBpB}$	pBpB: -5/7, nucpBpB: 20/9
10	1	tau: 1

2. Translation into Lean. Each rate term is a `KExpr`, for example `KExpr.npow (KExpr.var 0) 2` stands for $\mathrm{STAT5A}^2$; numbers are exact rationals, and the list `Rx` pairs each term with its stoichiometric column. The initial value is the rational vector `xq` (STAT5A and STAT5B positive, all other states 0), and θ_q holds the six parameters.

3. Checks. Four Boolean checks are evaluated by the Lean kernel: `net_ok` (every rate term is well defined for $x \geq 0$, and every term that consumes a species vanishes when that species is 0, so the field is quasi-positive), `growth_ok` (with weights $w_i = 1$, $\sum_i f_i \leq a + b \sum_i x_i$: the stoichiometric columns of six terms have a negative sum, and the remaining ones, the nuclear-import terms 7–9 and $\dot{\tau} = 1$, are at most linear in the states), `theta_ok` (all $\theta_j > 0$) and `x0_ok` ($x_q \geq 0$).

4. Final theorem. `def final := network_final pos Rx net_ok c growth_ok` θ_q `theta_ok xq x0_ok` applies the end-to-end theorem of step 28 to these data. Its statement is: for every $T \geq 0$ there is a solution x_0 on $[0, T]$ with $x_0(0) = x_q$, $x_0(t) \geq 0$ and $(x_0(t), \theta_q)$ in the domain; there is a family $x(\cdot; \theta)$ of solutions for θ near θ_q with $x(\cdot; \theta_q) = x_0$; and $x(t; \theta)$ is differentiable in θ at θ_q with derivative $S(t)$, $S(0) = 0$, solving the variational equation. These are the hypotheses of Parts II–III for this model.

5. Reproduction. From the folder `lean_proofs/`, the command

```
lake env lean Models/Boehm_JProteomeRes2014.lean
```

compiles the file in a few seconds and prints

```
'Models.Boehm_JProteomeRes2014.final' depends on axioms:  [propext,
Classical.choice, Quot.sound]
```

and, from the folder `certificados/`, the command

```
python3 verifica_traduccion.py Boehm_JProteomeRes2014
```

compares the file with the SBML model (largest relative difference 4×10^{-16}).

Benchmark problems without a verified model

Of the 34 benchmark problems analysed in the paper, 27 have a verified model; Froehlich, which is verified in Lean, is the further problem analysed only on its state trajectories. The remaining seven problems were not verified, for the following reasons. They are not counted in any statement that relies on G5.

- **Smith**: two reactions use $\max(\mathrm{PIP3} - \mathrm{basal}, 0)$, which is not differentiable, so the vector field is not C^1 and the sensitivity theorem does not apply.
- **Liu** and **Oliveira**: they contain events triggered by the state or by a parameter-dependent condition, not at fixed times; such events are outside Hypothesis 2 (step 18).
- **Alkan**: the model translates, but the syntactic checkers cannot establish well-definedness for 22 rate terms or quasi-positivity for 6 of them; this does not mean that the properties fail, only that the automatic proof does not cover them.
- **Perelson** and **Schwen**: the quasi-positivity check fails for two rate terms each, so non-negativity of the solutions is not established by the checker.
- **Beer**: smoothness and quasi-positivity are verified on both segments of the model, but the linear growth check is not satisfied, so existence of the nominal solution on the horizon is not proved; only a conditional theorem (assuming the solution exists) is available.

Data of each model

Armistead_CellDeathDis2024

- **File:** `Models/Armistead_CellDeathDis2024.lean`.
- **States ($n = 4$):** Cer, Sphingo, S1P, Sphinga.
- **Strictly positive species Σ (4):** Cer, Sphingo, S1P, Sphinga.
- **Parameters and nominal values θ_0 ($p = 10$):** k00 = 0.2381; k0 = 1000; k_d = 0.9068; k1 = 0.3011; k2 = 1000; k3 = 0.001; k4 = 2.362; k5 = 0.001001; alpha_hai1a = 0.5; alpha_cer = -0.3275.
- **Parameters declared positive:** 9 of 10.
- **Rate terms:** 8, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).

- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the strict region P_Σ, quasi-positive, and consumption of species in Σ at most proportional to their concentration (`checkNetS`); linear growth bound on P_Σ (`checkGrowthS`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta_0) \in P_\Sigma$ (`checkInitS`); declared parameter bounds hold at θ_0 (`checkUB`).
- **Fidelity of the translation:** largest relative difference 0.
- **Final theorem** (`Models.Armistead_CellDeathDis2024.final` := `strict_final_init`): for every $T \geq 0$ the nominal solution exists on $[0, T]$ and stays in P_Σ (species in Σ strictly positive) and in the domain; solutions near θ_0 exist and are differentiable, $S(0) = \partial_\theta G(\theta_0)$.

Bachmann_MSB2011

- **File:** `Models/Bachmann_MSB2011.lean`.
- **States ($n = 25$):** EpoRJAK2, EpoRpJAK2, p1EpoRpJAK2, p2EpoRpJAK2, p12EpoRpJAK2, EpoRJAK2_CIS, SHP1, SHP1Act, STAT5, pSTAT5, npSTAT5, CISnRNA1, CISnRNA2, CISnRNA3, CISnRNA4, CISnRNA5, CISRNA, CIS, SOCS3nRNA1, SOCS3nRNA2, SOCS3nRNA3, SOCS3nRNA4, SOCS3nRNA5, SOCS3RNA, SOCS3.
- **Parameters and nominal values θ_0 ($p = 27$):** CISEqc = 432.9; CISEqcOE = 0.5303; CISInh = 7.852×10^8; CISRNADelay = 0.1448; CISRNATurn = 1000; CISTurn = 0.008399; EpoRActJAK2 = 0.2673; EpoRCISInh = 1×10^6; EpoRCISRemove = 5.43; JAK2ActEpo = 6.332×10^5; JAK2EpoRDeaSHP1 = 142.7; SHP1ActEpoR = 0.001; SHP1Dea = 0.008162; SHP1ProOE = 2.826; SOCS3Eqc = 173.6; SOCS3EqcOE = 0.6792; SOCS3Inh = 10.41; SOCS3RNADelay = 1.065; SOCS3RNATurn = 0.008309; SOCS3Turn = 1×10^4; STAT5ActEpoR = 39; STAT5ActJAK2 = 0.07811; STAT5Exp = 0.07451; STAT5Imp = 0.02689; init_EpoRJAK2 = 3.976; init_SHP1 = 26.73; init_STAT5 = 79.75.
- **Parameters declared positive:** 27 of 27.
- **Rate terms:** 43, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \leq a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta)$ well defined and non-negative at θ_0 (`checkInit`).
- **Fidelity of the translation:** largest relative difference 3×10^{-15}.
- **Final theorem** (`Models.Bachmann_MSB2011.final` := `network_final_init`): same as `network_final`, with initial value $x(0) = G(\theta)$ and $S(0) = \partial_\theta G(\theta_0)$.

Bertozzi_PNAS2020

- **File:** `Models/Bertozzi_PNAS2020.lean`.
- **States ($n = 3$):** I_, R_, S_.
- **Parameters and nominal values θ_0 ($p = 0$):** .
- **Parameters declared positive:** 0 of 0.
- **Rate terms:** 2, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (2 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \leq a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).

- **Fidelity of the translation:** largest relative difference 0.
- **Final theorem** (`Models.Bertozzi_PNAS2020.final := network_final`): for every $T \geq 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Blasi_CellSystems2016

- **File:** `Models/Blasi_CellSystems2016.lean`.
- **States ($n = 16$):** x_0ac, x_k8, x_k5, x_k12, x_k16, x_k5k8, x_k5k12, x_k5k16, x_k8k12, x_k8k16, x_k12k16, x_k5k8k12, x_k5k8k16, x_k5k12k16, x_k8k12k16, x_4ac.
- **Parameters and nominal values θ_0 ($p = 8$):** a_basal = 0.0668; a_k8 = 0.0273; a_k5_k5k12 = 2.062; a_k12_k5k12 = 0.5519; a_k16_k12k16 = 0.6959; a_k5k12_k5k8k12 = 0.3253; a_k12k16_k8k12k16 = 2.205; a_k8k12k16_4ac = 3.592.
- **Parameters declared positive:** 8 of 8.
- **Rate terms:** 36, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (1 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \leq a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 4×10^{-15}.
- **Final theorem** (`Models.Blasi_CellSystems2016.final := network_final`): for every $T \geq 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Boehm_JProteomeRes2014

- **File:** `Models/Boehm_JProteomeRes2014.lean`.
- **States ($n = 9$):** STAT5A, STAT5B, pApB, pApA, pBpB, nucpApA, nucpApB, nucpBpB, τ (tiempo).
- **Parameters and nominal values θ_0 ($p = 6$):** Epo_degradation_BaF3 = 0.02698; k_exp_hetero = 1.001×10^{-5}; k_exp_homo = 0.00617; k_imp_hetero = 0.01637; k_imp_homo = 9.775×10^4; k_phos = 1.577×10^4.
- **Parameters declared positive:** 6 of 6.
- **Rate terms:** 10, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (2 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \leq a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 4×10^{-16}.
- **Final theorem** (`Models.Boehm_JProteomeRes2014.final := network_final`): for every $T \geq 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Borghans_BiophysChem1997

- **File:** `Models/Borghans_BiophysChem1997.lean`.
- **States ($n = 3$):** Z_state, Y_state, A_state.
- **Strictly positive species Σ (3):** Z_state, Y_state, A_state.
- **Parameters and nominal values θ_0 ($p = 20$):** K2 = 0.09995; K_par = 11.41; Ka = 0.1976; Kd = 0.3925; Kf = 1.138; Kp = 0.9966; Ky = 0.2004; Kz = 0.3034; Vd = 92.87; Vm2 = 7.457; Vm3 = 22.7; Vp = 2.757; beta_par = 1.124; epsilon_par = 0.1632; init_A_state = 1; init_Y_state = 0.9993; init_Z_state = 0.08792; n_par = 4.103; v0 = 2.318; v1 = 1.005.
- **Parameters declared positive:** 20 of 20.
- **Rate terms:** 9, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the strict region P_Σ, quasi-positive, and consumption of species in Σ at most proportional to their concentration (`checkNetS`); linear growth bound on P_Σ (`checkGrowthS`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta_0) \in P_\Sigma$ (`checkInitS`); declared parameter bounds hold at θ_0 (`checkUB`).
- **Fidelity of the translation:** largest relative difference 2×10^{-16}.
- **Final theorem** (`Models.Borghans_BiophysChem1997.final` := `strict_final_init`): for every $T \geq 0$ the nominal solution exists on $[0, T]$ and stays in P_Σ (species in Σ strictly positive) and in the domain; solutions near θ_0 exist and are differentiable, $S(0) = \partial_\theta G(\theta_0)$.

Brannmark_JBC2010

- **File:** `Models/Brannmark_JBC2010.lean`.
- **States ($n = 9$):** IR, IRins, IRp, IRiP, IRi, IRS, IRSiP, X, Xp.
- **Parameters and nominal values θ_0 ($p = 15$):** k1a = 0.1772; k1aBasic = 0.0003928; k1b = 0.1747; k1c = 0.05086; k1d = 5×10^5; k1e = 1×10^{-6}; k1f = 5×10^5; k1g = 1729; k1r = 0.0267; k21 = 2.102; k22 = 666.8; k3 = 4.944×10^{-5}; k_IRSiP_DosR = 37.94; km2 = 1.16; km3 = 0.4164.
- **Parameters declared positive:** 15 of 15.
- **Rate terms:** 14, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (9 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \leq a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 1×10^{-13}.
- **Final theorem** (`Models.Brannmark_JBC2010.final` := `network_final`): for every $T \geq 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t; \theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Bruno_JExpBot2016

- **File:** `Models/Bruno_JExpBot2016.lean`.
- **States ($n = 7$):** bcar, bcry, b10, bio, ohb10, ohbio, zea.
- **Parameters and nominal values θ_0 ($p = 6$):** k5 = 0.003087; kb1 = 0.01663; kb2 = 0.005844; kc1 = 0.001679; kc2 = 0.006976; kc4 = 0.006083.

- **Parameters declared positive:** 6 of 6.
- **Rate terms:** 6, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (5 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 2×10^{-16}.
- **Final theorem** (`Models.Bruno_JExpBot2016.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Chen_MSB2009

- **File:** `Models/Chen_MSB2009.lean` and 5 part files in `Models/Chen_MSB2009/`.
- **States:** $n = 501$ species (listed in the file).
- **Parameters:** $p = 152$, with nominal values in θ_q (exact rationals) in the file.
- **Parameters declared positive:** 152 of 152.
- **Rate terms:** 1,243, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (27 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 8×10^{-14}.
- **Final theorem** (`Models.Chen_MSB2009.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Crauste_CellSystems2017

- **File:** `Models/Crauste_CellSystems2017.lean`.
- **States ($n = 5$):** Naive, EarlyEffector, LateEffector, Memory, Pathogen.
- **Parameters and nominal values θ_0 ($p = 12$):** delta_EL = 0.5179; delta_LM = 0.02258; delta_NE = 0.01193; mu_EE = 3.914×10^{-5}; mu_LE = 1×10^{-10}; mu_LL = 8.115×10^{-6}; mu_N = 0.7399; mu_P = 1×10^{-5}; mu_PE = 1.412×10^{-10}; mu_PL = 3.634×10^{-5}; rho_E = 0.5074; rho_P = 0.1264.
- **Parameters declared positive:** 12 of 12.
- **Rate terms:** 12, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (2 non-zero entries).
- **Weights w_i of the growth bound:** values 1, 1×10^5.
- **Horizon:** $T \le T_q = 1$ (quadratic growth; Riccati bound).
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`); quadratic growth bound $\sum_i w_i f_i \le Q(\varphi+1)^2$ and $1.1\, Q(\varphi(x_q)+1)T_q < 1$ (`checkRiccati`).
- **Fidelity of the translation:** largest relative difference 0.

- **Final theorem** (`Models.Crauste_CellSystems2017.final := riccati_final`): for every $T \le T_q$ the nominal solution exists on $[0, T]$, is non-negative and stays in the domain; solutions near θ_0 exist and are differentiable, $S(0) = 0$.

Elowitz_Nature2000

- **File:** `Models/Elowitz_Nature2000.lean`.
- **States ($n = 8$):** X_protein, Y_protein, Z_protein, GFP, X_mRNA, Y_mRNA, Z_mRNA, GFP_mRNA.
- **Strictly positive species Σ (8):** X_protein, Y_protein, Z_protein, GFP, X_mRNA, Y_mRNA, Z_mRNA, GFP_mRNA.
- **Parameters and nominal values θ_0 ($p = 18$):** KM $= 1 \times 10^{-5}$; eff $= 0.03016$; eff_GFP $= 0.001952$; init_GFP $= 3.387 \times 10^{-5}$; init_GFP_mRNA $= 131.4$; init_X_mRNA $= 2.557$; init_X_protein $= 30.81$; init_Y_mRNA $= 1000$; init_Y_protein $= 0.0008183$; init_Z_mRNA $= 19.37$; init_Z_protein $= 994.4$; n_Hill $= 1.523$; tau_mRNA $= 12.62$; tau_mRNA_GFP $= 0.0004041$; tau_prot $= 5.359$; tau_prot_GFP $= 71.93$; tps_active $= 0.6123$; tps_repr $= 1 \times 10^{-5}$.
- **Parameters declared positive:** 18 of 18.
- **Rate terms:** 18, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the strict region P_Σ, quasi-positive, and consumption of species in Σ at most proportional to their concentration (`checkNetS`); linear growth bound on P_Σ (`checkGrowthS`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta_0) \in P_\Sigma$ (`checkInitS`); declared parameter bounds hold at θ_0 (`checkUB`).
- **Fidelity of the translation:** largest relative difference 2×10^{-16}.
- **Final theorem** (`Models.Elowitz_Nature2000.final := strict_final_init`): for every $T \ge 0$ the nominal solution exists on $[0, T]$ and stays in P_Σ (species in Σ strictly positive) and in the domain; solutions near θ_0 exist and are differentiable, $S(0) = \partial_\theta G(\theta_0)$.

Fiedler_BMCSystBiol2016

- **File:** `Models/Fiedler_BMCSystBiol2016.lean`.
- **States ($n = 7$):** RAF, pRAF, MEK, pMEK, ERK, pERK, τ (tiempo).
- **Strictly positive species Σ (6):** RAF, pRAF, MEK, pMEK, ERK, pERK.
- **Parameters and nominal values θ_0 ($p = 12$):** K_1 $= 1 \times 10^{-5}$; K_2 $= 1.511$; K_3 $= 0.9605$; k10 $= 0.02641$; k11 $= 163.2$; k2 $= 1.226$; k3 $= 769.2$; k4 $= 2.907$; k5 $= 577.6$; k6 $= 3.608$; tau1 $= 0.2428$; tau2 $= 0.02114$.
- **Parameters declared positive:** 12 of 12.
- **Rate terms:** 8, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the strict region P_Σ, quasi-positive, and consumption of species in Σ at most proportional to their concentration (`checkNetS`); linear growth bound on P_Σ (`checkGrowthS`); nominal parameters positive where declared (`checkPosParams`); initial values $G(\theta_0) \in P_\Sigma$ certified by rational interval arithmetic (`checkInitI`); declared parameter bounds hold at θ_0 (`checkUB`); the real powers in G are square roots.
- **Fidelity of the translation:** largest relative difference 0.
- **Final theorem** (`Models.Fiedler_BMCSystBiol2016.final := strict_final_initI`): same as `strict_final_init`, with the initial values certified by interval arithmetic.

Froehlich_CellSystems2018

- **File:** `Models/Froehlich_CellSystems2018.lean` and 13 part files in `Models/Froehlich_CellSystems2018/`.
- **States:** $n = 1{,}228$ species (listed in the file).
- **Parameters:** $p = 4088$, with nominal values in θ_q (exact rationals) in the file.
- **Parameters declared positive:** 4088 of 4088.
- **Rate terms:** 4,487, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (0 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference pending.
- **Final theorem** (`Models.Froehlich_CellSystems2018.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Fujita_SciSignal2010

- **File:** `Models/Fujita_SciSignal2010.lean`.
- **States ($n = 9$):** EGFR, pEGFR, pEGFR_Akt, Akt, pAkt, S6, pAkt_S6, pS6, EGF_EGFR.
- **Parameters and nominal values θ_0 ($p = 16$):** EGFR_turnover = 0.00148; init_AKT = 0.02693; init_EGFR = 3.754×10^7; init_S6 = 188.1; reaction_1_k1 = 0.003691; reaction_1_k2 = 0.002301; reaction_2_k1 = 0.0009365; reaction_2_k2 = 6.097×10^4; reaction_3_k1 = 0.4332; reaction_4_k1 = 0.03016; reaction_5_k1 = 3.273×10^{-6}; reaction_5_k2 = 0.0003985; reaction_6_k1 = 5.463×10^{-6}; reaction_7_k1 = 0.0118; reaction_8_k1 = 0.0009448; reaction_9_k1 = 0.02851.
- **Parameters declared positive:** 16 of 16.
- **Rate terms:** 13, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta)$ well defined and non-negative at θ_0 (`checkInit`).
- **Fidelity of the translation:** largest relative difference 1×10^{-16}.
- **Final theorem** (`Models.Fujita_SciSignal2010.final` := `network_final_init`): same as `network_final`, with initial value $x(0) = G(\theta)$ and $S(0) = \partial_\theta G(\theta_0)$.

Giordano_Nature2020

- **File:** `Models/Giordano_Nature2020.lean`.
- **States ($n = 10$):** Susceptible, Infected, Diagnosed, Ailing, Recognized, Threatened, Healed, Extinct, CumulativeDiagnosed, DiagnosedHealed.
- **Parameters and nominal values θ_0 ($p = 43$):** alpha_0 = 0.57; beta_0 = 0.011; delta_0 = 0.011; epsilon_0 = 0.171; eta_0 = 0.125; gamma_0 = 0.456; kappa_0 = 0.017; lam_0 = 0.034; mu_0 = 0.017; nu_0 = 0.027; rho_0 = 0.034; sigma_0 = 0.017; tau = 0.01; theta = 0.371; xi_0 = 0.017; zeta_0 = 0.125; alpha_4 = 0.422;

alpha_22 = 0.36; alpha_28 = 0.21; beta_4 = 0.0057; beta_22 = 0.005; delta_4 = 0.0057; delta_22 = 0.005; epsilon_12 = 0.143; epsilon_38 = 0.2; eta_22 = 0.034; eta_38 = 0.025; gamma_4 = 0.285; gamma_22 = 0.2; gamma_28 = 0.11; kappa_22 = 0.017; kappa_38 = 0.02; lam_22 = 0.08; mu_22 = 0.008; nu_22 = 0.015; rho_22 = 0.017; rho_38 = 0.02; sigma_22 = 0.017; sigma_38 = 0.01; xi_22 = 0.017; xi_38 = 0.02; zeta_22 = 0.034; zeta_38 = 0.025.

- **Parameters declared positive:** 43 of 43.
- **Rate terms:** 96, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (6 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Segments:** [0, 4], [4, 12], [12, 22], [22, 28], [28, 38], [38, 50].
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`); segment lengths are non-negative.
- **Fidelity of the translation:** largest relative difference 4×10^{-16}.
- **Final theorem** (`Models.Giordano_Nature2020.final` := `network_segments_final`): the same conclusions on each segment of the piecewise-defined model, with continuity across segment boundaries.

Isensee_JCB2018

- **File:** `Models/Isensee_JCB2018.lean`.
- **States (n = 25):** AC, pAC, AC_Fsk, pAC_Fsk, cAMP, Csub, Csub_H89, RII_C_2, RIIp_C_2, RIIp_cAMP_C_2, RIIp_cAMP_2, Sp8_Br_cAMPS, Rp8_Br_cAMPS, Rp8_pCPT_cAMPS, Rp_cAMPS, RIIp_Rp8_Br_cAMPS_C_2, RIIp_Rp8_pCPT_cAMPS_C_2, RIIp_Rp_cAMPS_C_2, RIIp_Sp8_Br_cAMPS_C_2, RIIp_Sp8_Br_cAMPS_2, RIIp_2, RII_2, IBMX, PDE, pPDE.
- **Parameters and nominal values θ_0 (p = 32):** KD_Fsk = 14.28; KD_H89 = 0.04807; KD_IBMX = 11.96; KD_cAMP = 2.85; kdeg_cAMP = 1×10^{-5}; kdeg_cAMP_free = 7.384; kf_Fsk = 1000; kf_H89 = 0.0007427; kf_RII_2__RII_C_2 = 1.264; kf_RII_C_2__RII_2 = 0.01875; kf_RII_C_2__RIIp_C_2 = 0.02277; kf_RIIp_2__RII_2 = 0.0323; kf_RIIp_C_2__RII_C_2 = 0.0257; kf_RIIp_cAMP_C_2__RIIp_2 = 0.1053; kf_cAMP = 0.5015; ki_IBMX = 1000; ki_Rp8_Br_cAMPS_pAB = 6.446; ki_Rp8_pCPT_cAMPS_pAB = 9.39; ki_Rp_cAMPS_pAB = 0.01632; ki_Sp8_Br_cAMPS_AM = 0.1317; ks_AC_cAMP = 0.4232; xi_AC_cAMP_Fsk = 642.4; xi_b_Rp8_Br_cAMPS = 0.04056; xi_b_Rp8_pCPT_cAMPS = 0.02668; xi_b_Rp_cAMPS = 0.6039; xi_b_Sp8_Br_cAMPS = 21.62; xi_kf_RII_2__RII_C_2 = 0.01893; xi_kf_RII_C_2__RII_2 = 0.811; xi_KD_Rp8_Br_cAMPS = 0.03996; xi_KD_Rp8_pCPT_cAMPS = 0.1942; xi_KD_Rp_cAMPS = 0.163; xi_KD_Sp8_Br_cAMPS = 0.2232.
- **Parameters declared positive:** 32 of 32.
- **Rate terms:** 34, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (10 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 1×10^{-14}.
- **Final theorem** (`Models.Isensee_JCB2018.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Lang_PLOSComputBiol2024

- **File:** `Models/Lang_PLOSComputBiol2024.lean`.
- **States:** $n = 124$ species (listed in the file).
- **Parameters:** $p = 164$, with nominal values in θ_q (exact rationals) in the file.
- **Parameters declared positive:** 164 of 164.
- **Rate terms:** 663, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (107 non-zero entries).
- **Weights w_i of the growth bound:** values 1, 2, 3.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 2×10^{-15}.
- **Final theorem** (`Models.Lang_PLOSComputBiol2024.final := network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Laske_PLOSComputBiol2019

- **File:** `Models/Laske_PLOSComputBiol2019.lean`.
- **States ($n = 34$):** V_ex, V_end, Vp_cyt, Vp_nuc, R_C, R_C_RdRp, R_V, R_V_RdRp, R_M1, R_M2, R_M3, R_M4, R_M5, R_M6, R_M7, R_M8, P_B1, P_B2, P_PA, P_HA, P_NP, P_NA, P_M1, P_NEP, Vp_nuc_M1, Vp_cyt_M1, V_rel, P_RdRp, Cp, V_att_Hi, V_att_Lo, P_M2, B_att_Lo, B_att_Hi.
- **Parameters and nominal values θ_0 ($p = 6$):** k_imp = 0.296; k_syn_R_M = 3.06×10^4; k_syn_R_C = 1.53; k_syn_R_V = 100.9; k_bind_M1 = 1.82×10^{-6}; k_rel = 0.0011.
- **Parameters declared positive:** 6 of 6.
- **Rate terms:** 51, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (3 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 0.
- **Final theorem** (`Models.Laske_PLOSComputBiol2019.final := network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Lucarelli_CellSystems2018

- **File:** `Models/Lucarelli_CellSystems2018.lean`.
- **States ($n = 33$):** TGFb, Rec, TGFb_pRec, S2, S3, S4, S2_S4_S4, ppS2_ppS2_ppS2, ppS3_ppS3_ppS3, S4_S4_S4, pS2, pS3, ppS2, ppS3, ppS2_ppS2_S4, ppS2_ppS2_ppS3, ppS2_ppS3_ppS3, ppS3_ppS3_S4, ppS2_ppS3_S4, ppS3_S4_S4, ppS2_S4_S4, geneA, geneB, geneC, geneD, geneE, geneF, geneG, geneH, geneI, geneJ, geneK, geneL.
- **Parameters:** $p = 69$, with nominal values in θ_q (exact rationals) in the file.
- **Parameters declared positive:** 69 of 69.

- **Rate terms:** 67, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta)$ well defined and non-negative at θ_0 (`checkInit`).
- **Fidelity of the translation:** largest relative difference 4×10^{-15}.
- **Final theorem** (`Models.Lucarelli_CellSystems2018.final := network_final_init`): same as `network_final`, with initial value $x(0) = G(\theta)$ and $S(0) = \partial_\theta G(\theta_0)$.

Okuonghae_ChaosSolitonsFractals2020

- **File:** `Models/Okuonghae_ChaosSolitonsFractals2020.lean`.
- **States ($n = 8$):** susceptible, exposed, asymptomatic, symptomatic, detected, recovered, deceased, detected_cumulative.
- **Strictly positive species Σ (6):** susceptible, exposed, asymptomatic, symptomatic, detected, detected_cumulative.
- **Parameters and nominal values θ_0 ($p = 14$):** alpha = 0.5; d_0 = 0.015; d_D = 0.015; gamma_0 = 0.1398; gamma_a = 0.1398; gamma_i = 0.06667; nu = 0.5; psi = 0.0135; sigma = 0.1923; theta = 2×10^{-12}; asymptomatic_start = 188; symptomatic_start = 212; exposed_start = 441; transmission_rate_effective = 0.4236.
- **Parameters declared positive:** 14 of 14.
- **Rate terms:** 11, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the strict region P_Σ, quasi-positive, and consumption of species in Σ at most proportional to their concentration (`checkNetS`); linear growth bound on P_Σ (`checkGrowthS`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta_0) \in P_\Sigma$ (`checkInitS`); declared parameter bounds hold at θ_0 (`checkUB`).
- **Fidelity of the translation:** largest relative difference 7×10^{-16}.
- **Final theorem** (`Models.Okuonghae_ChaosSolitonsFractals2020.final := strict_final_init`): for every $T \ge 0$ the nominal solution exists on $[0, T]$ and stays in P_Σ (species in Σ strictly positive) and in the domain; solutions near θ_0 exist and are differentiable, $S(0) = \partial_\theta G(\theta_0)$.

Rahman_MBS2016

- **File:** `Models/Rahman_MBS2016.lean`.
- **States ($n = 7$):** susceptible, infected_normal, infected_moderate, infected_weak, treated_normal, treated_moderate, treated_weak.
- **Strictly positive species Σ (4):** susceptible, infected_normal, infected_moderate, infected_weak.
- **Parameters and nominal values θ_0 ($p = 9$):** infected_normal_transmission_rate_relative = 427.5; infected_moderate_transmission_rate = 0.002927; infected_weak_transmission_rate_relative = 993; infected_weak_treatment_rate = 0.0001; infected_normal_worsen_rate = 0.6654; infected_moderate_worsen_rate = 0.007976; treated_moderate_improve_rate = 0.1364; treated_weak_improve_rate = 0.7756; behavioural_change_rate = 1.338×10^{-7}.
- **Parameters declared positive:** 9 of 9.

- **Rate terms:** 19, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the strict region P_Σ, quasi-positive, and consumption of species in Σ at most proportional to their concentration (`checkNetS`); linear growth bound on P_Σ (`checkGrowthS`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta_0) \in P_\Sigma$ (`checkInitS`); declared parameter bounds hold at θ_0 (`checkUB`).
- **Fidelity of the translation:** largest relative difference 1×10^{-15}.
- **Final theorem** (`Models.Rahman_MBS2016.final` := `strict_final_init`): for every $T \geq 0$ the nominal solution exists on $[0, T]$ and stays in P_Σ (species in Σ strictly positive) and in the domain; solutions near θ_0 exist and are differentiable, $S(0) = \partial_\theta G(\theta_0)$.

Raia_CancerResearch2011

- **File:** `Models/Raia_CancerResearch2011.lean`.
- **States ($n = 14$):** Rec, Rec_i, IL13_Rec, p_IL13_Rec, p_IL13_Rec_i, JAK2, pJAK2, STAT5, pSTAT5, SOCS3mRNA, DecoyR, IL13_DecoyR, SOCS3, CD274mRNA.
- **Parameters and nominal values θ_0 ($p = 18$):** CD274mRNA_production = 0.02121; DecoyR_binding = 0.003851; JAK2_p_inhibition = 0.07814; JAK2_phosphorylation = 1.017; Kon_IL13Rec = 0.002278; Rec_intern = 0.3441; Rec_phosphorylation = 1000; Rec_recycle = 0.002098; SOCS3_accumulation = 451.8; SOCS3_degradation = 0.04338; SOCS3_translation = 15.72; SOCS3mRNA_production = 0.1434; STAT5_phosphorylation = 0.01948; init_Rec_i = 235.1; pJAK2_dephosphorylation = 0.0001759; pRec_degradation = 0.2096; pRec_intern = 0.5853; pSTAT5_dephosphorylation = 0.0002748.
- **Parameters declared positive:** 18 of 18.
- **Rate terms:** 14, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \leq a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta)$ well defined and non-negative at θ_0 (`checkInit`).
- **Fidelity of the translation:** largest relative difference 6×10^{-16}.
- **Final theorem** (`Models.Raia_CancerResearch2011.final` := `network_final_init`): same as `network_final`, with initial value $x(0) = G(\theta)$ and $S(0) = \partial_\theta G(\theta_0)$.

Raimundez_PCB2020

- **File:** `Models/Raimundez_PCB2020.lean`.
- **States ($n = 22$):** EGFR, EGFR_CET, EGFR_EGF, EGFR_EGF_2, pEGFR_EGF_2, pEGFR_EGF_2_i, MMET, MMET_2, pMMET_2, pMMET_2_i, MMET_EGFR, pMMET_pEGFR, pMMET_pEGFR_i, MMET_METinh, MMET_MMET_METinh, MMET_METinh_2, EGFR_MMET_METinh, RAS_GTP, pMAPK, pPI3K, pMPI3K, pAKT.
- **Parameters and nominal values θ_0 ($p = 57$):** KD_EGFR_CET = 0.218; KD_EGFR_EGF = 0.3208; KD_METinh = 0.04194; d_AKTtotal__MKN1_2_HS746T = 0; d_AKTtotal__fm_2_hm = 0.8229; d_MAPKtotal__MKN1_2_HS746T = 0; d_MAPKtotal__fm_2_hm = 0.4275; d_MPI3Ktotal__fm_2_hm = -1.545; d_PI3Ktotal__fm_2_hm = -0.5318; d_RAStotal__MKN1_2_HS746T = -1.06×10^{-22}; d_RAStotal__fm_2_hm = -0.5382; d_kdeg_membran__MKN1_2_HS746T = -0.6; d_kdeg_pEGFR_EGF_2_i__MKN1_2_HS746T = 3; d_kexp_pEGFR_EGF_2_i__MKN1_2_HS746T = -0.2382; d_kimp_pEGFR_EGF_2__MKN1_2_HS746T = -1.746; d_ksyn_EGFR__MKN1_2_HS746T = -3;

d_ksyn_EGFR__fm_2_hm = -0.0876; d_ksyn_MMET__fm_2_hm = -0.5189; ka_AKT__MKN1 = 443.1; ka_MAPK__MKN1 = 1000; ka_PI3K__basal = 0.0003649; ka_PI3K__pEGFR_EGF_2 = 0.001701; ka_RAS__basal__MKN1 = 0.0002145; ka_RAS__pEGFR_EGF_2__MKN1 = 1.47×10^{-5}; kbin_EGFR_CET = 0.5853; kbin_EGFR_EGF = 8.882; kdeg_membran__MKN1 = 0.002259; kdeg_pEGFR_EGF_2_i__MKN1 = 0.0668; kdim_EGFR_EGF = 0.03239; kdim_MMET = 2.61×10^{-5}; kdim_MMET_EGFR = 0.001882; kdim_MMETinh = 0.03127; kexp_pEGFR_EGF_2_i__MKN1 = 0.7517; ki_AKT__MKN1 = 18.47; ki_MAPK = 2.177; ki_PI3K__MKN1 = 1.316; ki_RAS__MKN1 = 0.327; kimp_pEGFR_EGF_2__MKN1 = 0.7469; kpho_EGFR_EGF = 1.344; kpho_MMET = 2.543; kpho_MMET_EGFR = 4.563; ksyn_MMET__HS746T_fm = 0.02635; xi_ka_PI3K_pMMET_2 = 301.3; xi_ka_PI3K_pMMET_pEGFR = 0.7655; xi_ka_RAS_pMMET_2 = 0.4494; xi_ka_RAS_pMMET_pEGFR = 0.2536; xi_kdeg_pMMET_2_i = 0.7837; xi_kdeg_pMMET_pEGFR_i = 3.533; xi_kdim_MMET = 0.9748; xi_kdim_MMET_EGFR = 1; xi_kexp_pMMET_2_i = 1.243; xi_kexp_pMMET_pEGFR_i = 0.2864; xi_ki_MPI3K = 0.00322; xi_kimp_pMMET_2 = 0.07715; xi_kimp_pMMET_pEGFR = 0.00796; xi_kpho_MMET = 0.008328; xi_kpho_MMET_EGFR = 0.06967.

- **Parameters declared positive:** 45 of 57.
- **Rate terms:** 91, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (0 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 1×10^{-12}.
- **Final theorem** (`Models.Raimundez_PCB2020.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

SalazarCavazos_MBoC2020

- **File:** `Models/SalazarCavazos_MBoC2020.lean`.
- **States:** $n = 75$ species (listed in the file).
- **Parameters and nominal values θ_0 ($p = 6$):** GRB2_total__FREE = 1.699×10^5; SHC1_total__FREE = 6.494×10^5; kdephosY1068__FREE = 1.659; kdephosYN__FREE = 0.01718; ratio_kpkd_Y1068__FREE = 0.1575; ratio_kpkd_YN__FREE = 0.4448.
- **Parameters declared positive:** 6 of 6.
- **Rate terms:** 457, each with its stoichiometric column.
- **Initial values:** expressions $G(\theta)$ in the parameters (SBML initial assignments).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); parameter-dependent initial values $G(\theta)$ well defined and non-negative at θ_0 (`checkInit`).
- **Fidelity of the translation:** largest relative difference 2×10^{-13}.
- **Final theorem** (`Models.SalazarCavazos_MBoC2020.final` := `network_final_init`): same as `network_final`, with initial value $x(0) = G(\theta)$ and $S(0) = \partial_\theta G(\theta_0)$.

Sneyd_PNAS2002

- **File:** `Models/Sneyd_PNAS2002.lean`.

- **States ($n = 6$):** IPR_O, IPR_R, IPR_I1, IPR_S, IPR_A, IPR_I2.
- **Parameters and nominal values θ_0 ($p = 14$):** k1 = 3.727; k2 = 1×10^5; k3 = 15.75; k4 = 9.994×10^4; k_1 = 0.9239; k_2 = 0.001002; k_3 = 1.915; k_4 = 3079; l2 = 0.9401; l4 = 2.858; l6 = 1×10^5; l_2 = 0.3477; l_4 = 0.01388; l_6 = 0.001.
- **Parameters declared positive:** 14 of 14.
- **Rate terms:** 16, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (1 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 4×10^{-15}.
- **Final theorem** (`Models.Sneyd_PNAS2002.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Weber_BMC2015

- **File:** `Models/Weber_BMC2015.lean`.
- **States ($n = 7$):** PKD, PKDDAGa, PI4K3B, PI4K3Ba, CERTERa, CERT, CERTTGNa.
- **Parameters and nominal values θ_0 ($p = 26$):** a11 = 0.1825; a12 = 26.67; a21 = 1.877; a22 = 0.0001; a31 = 0.1353; a32 = 0.0001; a33 = 0.0001; m11 = 1×10^{10}; m22 = 2154; m31 = 9.97×10^9; m33 = 4.187×10^7; p11 = 4.244; p12 = 0.005936; p13 = 0.00246; p21 = 5.629; p22 = 21.79; p31 = 2538; p32 = 16.77; p33 = 2.166×10^4; pu3 = 1×10^8; pu4 = 3.313×10^7; pu5 = 33.7; pu6 = 113.4; s12 = 8.846×10^4; s21 = 2.961×10^6; s31 = 4.328×10^6.
- **Parameters declared positive:** 26 of 26.
- **Rate terms:** 18, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (7 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 0.
- **Final theorem** (`Models.Weber_BMC2015.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Zhao_QuantBiol2020

- **File:** `Models/Zhao_QuantBiol2020.lean`.
- **States ($n = 4$):** Susceptible, Unquarantined_Infected, Quarantined_Infected, Confirmed_Infected.
- **Parameters and nominal values θ_0 ($p = 21$):** R_Stage_I_Wuhan = 4.709; gamma_1_Stage_I_Wuhan = 0.063; gamma_2_Stage_I_Wuhan = 0.05; R_Stage_II_Wuhan = 0.7575; gamma_1_Stage_II_Wuhan = 0.3917; gamma_2_Stage_II_Wuhan = 0.0643; R_Stage_III_Wuhan = 0.4797; gamma_1_Stage_III_Wuhan = 0.6185; gamma_2_Stage_III_Wuhan = 0.322; R_Stage_I_Hubei = 5.934; gamma_1_Stage_I_Hubei = 0.05; gamma_2_Stage_I_Hubei = 0.05; R_Stage_II_Hubei = 0.6079; gamma_1_Stage_II_Hubei = 0.488; gamma_2_Stage_II_Hubei = 0.1914; R_Stage_I_China = 1.528; gamma_1_Stage_I_China = 0.1941;

gamma_2_Stage_I_China = 0.05; R_Stage_II_China = 0.5753; gamma_1_Stage_II_China = 0.5157; gamma_2_Stage_II_China = 0.2189.

- **Parameters declared positive:** 21 of 21.
- **Rate terms:** 3, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (3 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 0.
- **Final theorem** (`Models.Zhao_QuantBiol2020.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Zheng_PNAS2012

- **File:** `Models/Zheng_PNAS2012.lean`.
- **States ($n = 15$):** K27me0K36me0, K27me0K36me1, K27me1K36me0, K27me0K36me2, K27me1K36me1, K27me2K36me0, K27me0K36me3, K27me1K36me2, K27me2K36me1, K27me3K36me0, K27me1K36me3, K27me2K36me2, K27me3K36me1, K27me2K36me3, K27me3K36me2.
- **Parameters and nominal values θ_0 ($p = 45$):** inflowp = 0.03048; k00_01 = 997.3; k00_10 = 0.234; k01_00 = 479.8; k01_02 = 1.913; k01_11 = 2.651; k02_01 = 0.06996; k02_03 = 1×10^{-5}; k02_12 = 1×10^{-5}; k03_02 = 1×10^{-5}; k03_13 = 988.1; k10_00 = 1×10^{-5}; k10_11 = 1×10^{-5}; k10_20 = 0.2692; k11_01 = 1×10^{-5}; k11_10 = 1×10^{-5}; k11_12 = 11.85; k11_21 = 1×10^{-5}; k12_02 = 1×10^{-5}; k12_11 = 0.107; k12_13 = 0.03563; k12_22 = 0.01062; k13_03 = 450.4; k13_12 = 0.1953; k13_23 = 1×10^{-5}; k20_10 = 1×10^{-5}; k20_21 = 1×10^{-5}; k20_30 = 0.1507; k21_11 = 1×10^{-5}; k21_20 = 0.0354; k21_22 = 999.3; k21_31 = 1×10^{-5}; k22_12 = 0.05338; k22_21 = 426.1; k22_23 = 0.1517; k22_32 = 1×10^{-5}; k23_13 = 0.7912; k23_22 = 1×10^{-5}; k30_20 = 1×10^{-5}; k30_31 = 0.2813; k31_21 = 1.001×10^{-5}; k31_30 = 1×10^{-5}; k31_32 = 0.3536; k32_22 = 0.633; k32_31 = 1×10^{-5}.
- **Parameters declared positive:** 45 of 45.
- **Rate terms:** 44, each with its stoichiometric column.
- **Initial values:** constant rational vector x_q (15 non-zero entries).
- **Weights w_i of the growth bound:** all equal to 1.
- **Checks executed by the kernel:** rate laws well defined on the non-negative orthant and quasi-positive (`checkNet`); linear growth bound $\sum_i w_i f_i \le a + b \sum_i w_i x_i$ with the weights w (`checkGrowth`); nominal parameters positive where declared (`checkPosParams`); initial values non-negative (`checkNonneg`).
- **Fidelity of the translation:** largest relative difference 1×10^{-15}.
- **Final theorem** (`Models.Zheng_PNAS2012.final` := `network_final`): for every $T \ge 0$ the nominal solution from x_q exists on $[0, T]$, is non-negative and stays in the domain of the rate laws; solutions exist for all θ near θ_0, and $x(t;\theta)$ is differentiable in θ with sensitivity $S(t)$, $S(0) = 0$.

Supplementary Note 5. Independent reproduction of the verification

This note describes how a reader can check the formal verification without relying on our statements. Three different things can be checked, and they answer different questions.

What each check establishes

- *Compilation* establishes that every proof is accepted by the Lean kernel, i.e. that each theorem follows from its hypotheses.
- *Axiom inspection* (`#print axioms`) establishes that no proof relies on an unproved assumption: the only axioms are `propext`, `Classical.choice` and `Quot.sound`, which are part of Lean's standard logic.
- *The fidelity check* of Supplementary Note 4 establishes that the data on which the model checks run (rate laws, stoichiometry, parameter values and initial values) are those of the benchmark models.

The statements themselves should be compared with the mathematics they are meant to encode; Supplementary Note 3 gives each statement in mathematical notation for this purpose.

Versions

Lean 4.23.0 (file `lean_proofs/lean-toolchain`) and Mathlib at tag `v4.23.0` (file `lean_proofs/lakefile.lean`). The repository is `https://github.com/monicammr/archivos`; the version used for this article will be archived on Zenodo with a DOI upon acceptance, and the exact commit is stated there. The Python checks use Python 3.11, libRoadRunner and libSBML.

Steps

1. Install the Lean version manager `elan` (`https://github.com/leanprover/elan`); it reads the Lean version from `lean-toolchain`.
2. In the folder `lean_proofs/`, run `./verify.sh`. The script (i) checks that no file contains `sorry` or an `axiom` declaration, (ii) downloads the precompiled Mathlib (`lake exe cache get`), (iii) compiles the 31 theory modules and the 28 models (`lake build`) and (iv) prints the axioms of every theorem (`CheckAxioms.lean`) and of the final theorem of every model (`CheckModelAxioms.lean`). The theory compiles in a few minutes; the models take longer, the largest (Chen and Froehlich) about 30 minutes each. A large stack is needed (`ulimit -s unlimited`; the build passes `--tstack=4000000` to the model files).
3. To inspect one model only: `lake env lean Models/<model>.lean`. The last line of each model file prints the axioms of its final theorem.
4. To locate a theorem: Supplementary Table S3 gives the module of every main result, and `lean_proofs/THEOREMS.md` and `FORMULAS.md` list every declaration with its module; the module `X` is the file `lean_proofs/X.lean`.
5. To check the translation of the models: from the folder `certificados/`, `python3 verifica_traduccion.py` compares every model file with its SBML model and writes `resultados/sbml_lean/fidelidad.json`; a single model can be given as argument.

Our complete build log, with the compilation time of every module, is `lean_proofs/BUILD_LOG.md`.

Concluding remarks

This Supplementary Information was written so that every claim of the main text can be traced to its source and checked independently. Supplementary Tables S1 and S2 give the subsets selected for

each benchmark problem and the notation of the method, and Supplementary Note 1 compares our simulation of the measured outputs with the reference simulations of the benchmark. These materials connect the results reported in the paper with the data and code from which they were obtained.

The remaining material documents the formal verification and, just as importantly, its scope. Supplementary Figure S1 and Table S4 separate three levels of evidence: general mathematical guarantees proved in Lean 4 (G1–G4), hypotheses proved for specific models (G5), and numerical conditions computed in Python, on which the application of the guarantees to a particular result depends. Supplementary Table S3 and Note 2 follow the 31 modules of the formalisation in the order in which they depend on each other, from the energy bound on the error of the reduced model and the relation between $\cos\Delta$ and the best rescaled fit, through the differentiability of ODE solutions with respect to the parameters, to certificates for ODE models and the hypotheses of the benchmark models; they also state what the proofs do not establish, such as identifiability of the nonlinear models or the convergence of the numerical optimisation. Supplementary Note 3 and Table S5 give the statement and meaning of each of the 289 theorems and lemmas, so that every formal statement can be compared with the mathematics it encodes. Supplementary Note 4 and Table S6 show, model by model, the data on which the Lean kernel executed its checks, a complete worked example, an independent check that the translated equations agree with the SBML models, and the reasons why seven benchmark problems were not verified. Supplementary Note 5 explains how to reproduce all of this.

Taken together, the formalisation establishes that the steps of the method are mathematically sound under explicit hypotheses and that 28 benchmark models satisfy the hypotheses on the dynamics; the numerical quantities reported in the paper are computed in floating point by the accompanying Python code, and the formal results state precisely which conclusions follow from them. Everything described here can be reproduced from the repository: the proofs compile with a single command, the supplementary tables and notes are generated from the Lean sources and the result files, and the analyses can be rerun from the published benchmark. We hope that this combination of a public benchmark, open code and machine-checked mathematics, together with a clear statement of what each part guarantees, makes the reduction method easy to verify, to reuse and to extend to other dynamical models.